

2 Executing the Fregean Program

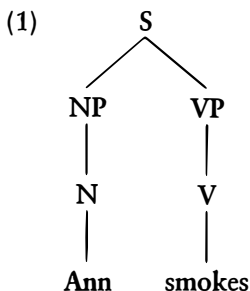
In the pages to follow, we will execute the Fregean program for a fragment of English. Although we will stay very close to Frege's proposals at the beginning, we are not interested in an exegesis of Frege, but in the systematic development of a semantic theory for natural language. Once we get beyond the most basic cases, there will be many small and some not-so-small departures from the semantic analyses that Frege actually defended. But his treatment of semantic composition as functional application (*Frege's Conjecture*), will remain a leading idea throughout.

Modern syntactic theory has taught us how to think about sentences and their parts. Sentences are represented as phrase structure trees. The parts of a sentence are subtrees of phrase structure trees. In this chapter, we begin to explore ways of interpreting phrase structure trees of the kind familiar in linguistics. We will proceed slowly. Our first fragment of English will be limited to simple intransitive and transitive sentences (with only proper names as subjects and objects), and extremely naive assumptions will be made about their structures. Our main concern will be with the process of meaning composition. We will see how a precise characterization of this process depends on, and in turn constrains, what we say about the interpretation of individual words.

This chapter, too, has sections which are not devoted to semantics proper, but to the mathematical tools on which this discipline relies. Depending on the reader's prior mathematical experience, these may be supplemented by exercises from other sources or skimmed for a quick review.

2.1 First example of a Fregean interpretation

We begin by limiting our attention to sentences that consist of a proper name plus an intransitive verb. Let us assume that the syntax of English associates these with phrase structures like that in (1).



We want to formulate a set of semantic rules which will provide denotations for all trees and subtrees in this kind of structure. How shall we go about this? What sorts of entities shall we employ as denotations? Let us be guided by Frege.

Frege took the denotations of sentences to be truth-values, and we will follow him in this respect. But wait. Can this be right? The previous chapter began with the statement “To know the meaning of a sentence is to know its truth-conditions”. We emphasized that the meaning of a sentence is not its actual truth-value, and concluded that a theory of meaning for natural language should pair sentences with their truth-*conditions* and explain how this can be done in a compositional way. Why, then, are we proposing truth-*values* as the denotations for sentences? Bear with us. Once we spell out the complete proposal, you’ll see that we will end up with truth-conditions after all.

The Fregean denotations that we are in the midst of introducing are also called “*extensions*”, a term of art which is often safer to use because it has no potentially interfering non-technical usage. The extension of a sentence, then, is its actual truth-value. What are truth-values? Let us identify them with the numbers 1 (True) and 0 (False). Since the extensions of sentences are not functions, they are saturated in Frege’s sense. The extensions of proper names like “Ann” and “Jan” don’t seem to be functions either. “Ann” denotes Ann, and “Jan” denotes Jan.

We are now ready to think about suitable extensions for intransitive verbs like “smokes”. Look at the above tree. We saw that the extension for the lexical item “Ann” is the individual Ann. The node dominating “Ann” is a non-branching N-node. This means that it should inherit the denotation of its daughter node.¹ The N-node is again dominated by a non-branching node. This NP-node, then, will inherit its denotation from the N-node. So the denotation of the NP-node in the above tree is the individual Ann. The NP-node is dominated by a branching S-node. The denotation of the S-node, then, is calculated from the denotation of the NP-node and the denotation of the VP-node. We know that the denotation of the NP-node is Ann, hence saturated. Recall now that Frege conjectured that all semantic composition amounts to functional application. If that is so, we

must conclude that the denotation of the VP-node must be unsaturated, hence a function. What kind of function? Well, we know what kinds of things its arguments and its values are. Its arguments are individuals like Ann, and its values are truth-values. The extension of an intransitive verb like “smokes”, then, should be a function from individuals to truth-values.

Let’s put this all together in an explicit formulation. Our semantics for the fragment of English under consideration consists of three components. First, we define our inventory of denotations. Second, we provide a lexicon which specifies the denotation of each item that may occupy a terminal node. Third, we give a semantic rule for each possible type of non-terminal node. When we want to talk about the denotation of a lexical item or tree, we enclose it in double brackets. For any expression α , then, $\llbracket \alpha \rrbracket$ is the denotation of α . We can think of $\llbracket \]$ as a function (the *interpretation function*) that assigns appropriate denotations to linguistic expressions. In this and most of the following chapters, the denotations of expressions are extensions. The resulting semantic system is an *extensional semantics*. Towards the end of this book, we will encounter phenomena that cannot be handled within an extensional semantics. We will then revise our system of denotations and introduce *intensions*.

A. Inventory of denotations

Let D be the set of all individuals that exist in the real world. Possible denotations are:

Elements of D , the set of actual individuals.

Elements of $\{0, 1\}$, the set of truth-values.

Functions from D to $\{0, 1\}$.

B. Lexicon

$\llbracket \text{Ann} \rrbracket = \text{Ann}$

$\llbracket \text{Jan} \rrbracket = \text{Jan}$

etc. for other proper names.

$\llbracket \text{works} \rrbracket = f : D \rightarrow \{0, 1\}$

For all $x \in D$, $f(x) = 1$ iff x works.

$\llbracket \text{smokes} \rrbracket = f : D \rightarrow \{0, 1\}$

For all $x \in D$, $f(x) = 1$ iff x smokes.

etc. for other intransitive verbs.

C. Rules for non-terminal nodes

In what follows, Greek letters are used as variables for trees and subtrees.

(S1) If α has the form $\begin{array}{c} S \\ \swarrow \searrow \\ \beta \quad \gamma \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \gamma \rrbracket(\llbracket \beta \rrbracket)$.

(S2) If α has the form $\begin{array}{c} NP \\ | \\ \beta \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket$.

(S3) If α has the form $\begin{array}{c} VP \\ | \\ \beta \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket$.

(S4) If α has the form $\begin{array}{c} N \\ | \\ \beta \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket$.

(S5) If α has the form $\begin{array}{c} V \\ | \\ \beta \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket$.

2.1.1 Applying the semantics to an example

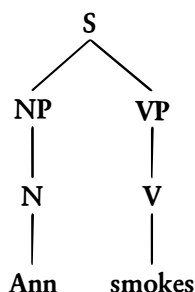
Does this set of semantic rules predict the correct truth-conditions for “Ann smokes”? That is, is “Ann smokes” predicted to be true if and only if Ann smokes? “Of course”, you will say, “that’s obvious”. It’s pretty obvious indeed, but we are still going to take the trouble to give an explicit proof of it. As matters get more complex in the chapters to come, it will be less and less obvious whether a given set of proposed rules predict the judgments it is supposed to predict. But you can always find out for sure if you draw your trees and work through them node by node, applying one rule at a time. It is best to get used to this while the calculations are still simple. If you have some experience with computations of this kind, you may skip this subsection.

We begin with a precise statement of the claim we want to prove:

Claim:

$$\left[\begin{array}{cc} & S \\ & \swarrow \searrow \\ NP & VP \\ | & | \\ N & V \\ | & | \\ \text{Ann} & \text{smokes} \end{array} \right] = 1 \text{ iff Ann smokes.}$$

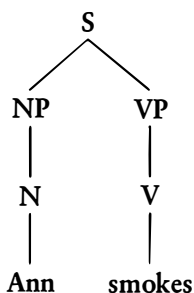
We want to deduce this claim from our lexical entries and semantic rules (S1)–(S5). Each of these rules refers to trees of a certain form. The tree



is of the form specified by rule (S1), repeated here, so let's see what (S1) says about it.

(S1) If α has the form $\begin{array}{c} S \\ \swarrow \searrow \\ \beta \quad \gamma \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \gamma \rrbracket(\llbracket \beta \rrbracket)$.

When we apply a general rule to a concrete tree, we must first match up the variables in the rule with the particular constituents that correspond to them in the application. In this instance, α is

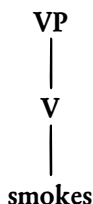


so β must be $\begin{array}{c} \text{NP} \\ | \\ \text{N} \\ | \\ \text{Ann} \end{array}$ and γ must be $\begin{array}{c} \text{VP} \\ | \\ \text{V} \\ | \\ \text{smokes} \end{array}$

The rule says that $\llbracket \alpha \rrbracket = \llbracket \gamma \rrbracket (\llbracket \beta \rrbracket)$, so this means in the present application that

$$(2) \left[\begin{array}{c} S \\ \swarrow \quad \searrow \\ NP \quad VP \\ | \quad | \\ N \quad V \\ | \quad | \\ Ann \quad smokes \end{array} \right] = \left[\begin{array}{c} VP \\ | \\ V \\ | \\ smokes \end{array} \right] \left(\left[\begin{array}{c} NP \\ | \\ N \\ | \\ Ann \end{array} \right] \right)$$

Now we apply rule (S3) to the tree



(This time, we skip the detailed justification of why and how this rule fits this tree.) What we obtain from this is

$$(3) \left[\begin{array}{c} VP \\ | \\ V \\ | \\ smokes \end{array} \right] = \left[\begin{array}{c} V \\ | \\ smokes \end{array} \right]$$

From (2) and (3), by substituting equals for equals, we infer (4).

$$(4) \left[\begin{array}{c} S \\ \swarrow \quad \searrow \\ NP \quad VP \\ | \quad | \\ N \quad V \\ | \quad | \\ Ann \quad smokes \end{array} \right] = \left[\begin{array}{c} V \\ | \\ smokes \end{array} \right] \left(\left[\begin{array}{c} NP \\ | \\ N \\ | \\ Ann \end{array} \right] \right)$$

Now we apply rule (S5) to the appropriate subtree and use the resulting equation for another substitution in (4):

$$(5) \left[\begin{array}{c} S \\ \swarrow \quad \searrow \\ NP \quad VP \\ | \quad | \\ N \quad V \\ | \quad | \\ \text{Ann} \quad \text{smokes} \end{array} \right] = \llbracket \text{smokes} \rrbracket \left[\begin{array}{c} NP \\ | \\ N \\ | \\ \text{Ann} \end{array} \right]$$

Now we use rule (S2) and then (S4), and after substituting the results thereof in (5), we have (6).

$$(6) \left[\begin{array}{c} S \\ \swarrow \quad \searrow \\ NP \quad VP \\ | \quad | \\ N \quad V \\ | \quad | \\ \text{Ann} \quad \text{smokes} \end{array} \right] = \llbracket \text{smokes} \rrbracket (\llbracket \text{Ann} \rrbracket)$$

At this point, we look up the lexical entries for **Ann** and **smokes**. If we just use these to substitute equals for equals in (6), we get (7).

$$(7) \left[\begin{array}{c} S \\ \swarrow \quad \searrow \\ NP \quad VP \\ | \quad | \\ N \quad V_i \\ | \quad | \\ \text{Ann} \quad \text{smokes} \end{array} \right] = \left[\begin{array}{l} f : D \rightarrow \{0, 1\} \\ \text{For all } x \in D, f(x) = 1 \text{ iff } x \text{ smokes} \end{array} \right] (\text{Ann})$$

Let's take a close look at the right-hand side of this equation. It has the gross form "function (argument)", so it denotes the value that a certain function yields for a certain argument. The argument is **Ann**, and the function is the one which maps those who smoke to 1 and all others to 0. If we apply this function to **Ann**, we will get 1 if **Ann** smokes and 0 if she doesn't. To summarize what we have just determined:

$$(8) \left[\begin{array}{l} f : D \rightarrow \{0, 1\} \\ \text{For all } x \in D, f(x) = 1 \text{ iff } x \text{ smokes} \end{array} \right] (\text{Ann}) = 1 \text{ iff Ann smokes.}$$

And now we have reached the goal of our proof: (7) and (8) together imply exactly the claim which we stated at the beginning. QED.

This was not the only way in which we could have constructed the proof of this claim. What matters is (a) that we use each applicable rule or lexical entry to obtain an equation regarding the denotation of a certain subtree; (b) that we keep using some of these equations to substitute equals for equals in others, thereby getting closer and closer to the target equation in our claim; and (c) that we employ the definitions of functions that we find in the lexicon to calculate their values for specified arguments. There is no unique specified order in which we must perform these steps. We can apply rules to the smallest subtrees first, or start at the top of the tree, or anywhere in the middle. We can collect a long list of separate equations before we begin to draw conclusions from any two of them, or else we can keep alternating applications of semantic rules with substitutions in equations derived previously. The soundness of the proof is not affected by these choices (although, of course, some strategies may be easier than others to follow through without getting confused).

We have used the word “proof” a number of times in this section. What exactly do we mean by this term? The notion of “proof” has been made precise in various ways in the history of logic. The most rigorous notion equates a proof with a syntactic derivation in an axiomatic or natural deduction system. Above, we relied on a less regimented notion of “proof” that is common in mathematics. Mathematical proofs are rarely algorithmic derivations. They are usually written in plain English (or some other natural language), supplemented by technical vocabulary that has been introduced through definitions. Conclusions are licensed by inference patterns that are known to be valid but are not spelled out formally. The proofs in this book are all “semi-formal” in this way. The standards of rigor followed in mathematics should be good enough for what we want to accomplish here.

2.1.2 *Deriving truth-conditions in an extensional semantics*

The proof we just gave illustrates how a semantic system based on extensions allows us to compute the truth-conditions, and hence the meaning, of a sentence. If you check the proof again, you will see that we end up with the truth-conditions of “Ann smokes” because the lexicon defines the extensions of predicates by specifying a *condition*. Had we defined the function denoted by “smokes” by displaying it in a table, for example, we would have obtained a

mere truth-*value*. We didn't really have a choice, though, because displaying the function in a table would have required more world knowledge than we happen to have. We do not know of every existing individual whether or not (s)he smokes. And that's certainly not what we have to know in order to know the meaning of "smoke". We could look at a fictitious example, though.

Suppose Ann, Jan, and Maria are the only individuals in the actual world, and Ann and Jan are the only smokers. The extension of the verb "smokes" can now be displayed in a table:

$$[[\text{smokes}]] = \begin{bmatrix} \text{Ann} \rightarrow 1 \\ \text{Jan} \rightarrow 1 \\ \text{Maria} \rightarrow 0 \end{bmatrix}$$

Using this way of defining the extension of "smokes", our computation would have ended as follows:

$$\left[\begin{array}{c} \text{S} \\ \swarrow \quad \searrow \\ \text{NP} \quad \text{VP} \\ | \quad | \\ \text{N} \quad \text{V} \\ | \quad | \\ \text{Ann} \quad \text{smokes} \end{array} \right] = \begin{bmatrix} \text{Ann} \rightarrow 1 \\ \text{Ann} \rightarrow 1 \\ \text{Maria} \rightarrow 0 \end{bmatrix} (\text{Ann}) = 1$$

Here, the sentence "Ann smokes" would not be paired with its truth-conditions, but with the value 1.

The issue of how an extensional system can yield a theory of meaning concerns the relationship between what Frege called "*Sinn*" and "*Bedeutung*". Frege's "*Bedeutung*" corresponds to our term "extension", and is sometimes translated as "reference".² Frege's "*Sinn*" is usually translated as "sense", and corresponds to what we have called "meaning". How does Frege get us from *Bedeutung* to *Sinn*? In his book on Frege's philosophy of language, Michael Dummett answers this question as follows:

It has become a standard complaint that Frege talks a great deal about the senses of expressions, but nowhere gives an account of what constitutes such a sense. This complaint is partly unfair: for Frege the sense of an expression is the manner in which we determine its reference, and he tells us a great deal about the kind of reference possessed by expressions of different types, thereby specifying the form that the senses of such expressions must

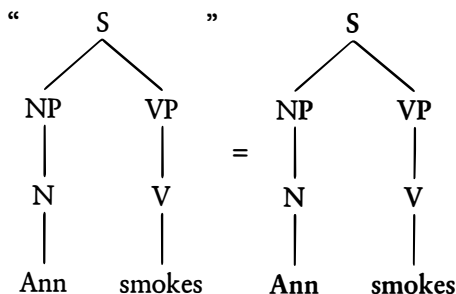
take. . . . The sense of an expression is the mode of presentation of the referent: in saying what the referent is, we have to choose a particular way of saying this, a particular means of determining something as a referent.³

What Dummett says in this passage is that when specifying the extension (reference, *Bedeutung*) of an expression, we have to choose a particular way of presenting it, and it is this manner of presentation that might be considered the meaning (sense, *Sinn*) of the expression. The function that is the extension of a predicate can be presented by providing a condition or by displaying it in a table, for example. Only if we provide a condition do we choose a mode of presentation that “shows”⁴ the meaning of the predicates and the sentences they occur in. Different ways of defining the same extensions, then, can make a theoretical difference. Not all choices yield a theory that pairs sentences with their truth-conditions. Hence not all choices lead to a theory of meaning.

2.1.3 Object language and metalanguage

Before we conclude this section, let us briefly reflect on a typographical convention that we have already been using. When we referred to words and phrases of English (represented as strings or trees), we replaced the customary quotes by bold-face. So we had, for example:

“Ann” = Ann.



The expressions that are bold-faced or enclosed in quotes are expressions of our *object language*, the language we are investigating. In this book, the object language is English, since we are developing a semantic theory for English. The language we use for theoretical statements is the *metalanguage*. Given that this book is written in English, our metalanguage is English as well. Since we are looking at the English object language in a fairly technical way, our English metalanguage includes a fair amount of technical vocabulary and notational

conventions. The abstraction notation for sets that we introduced earlier is an example. Quotes or typographical distinctions help us mark the distinction between object language and metalanguage. Above, we always used the bold-faced forms when we placed object language expressions between denotation brackets. For example, instead of writing the lexical entry for the name “Ann”:

$\llbracket \text{“Ann”} \rrbracket = \text{Ann}$

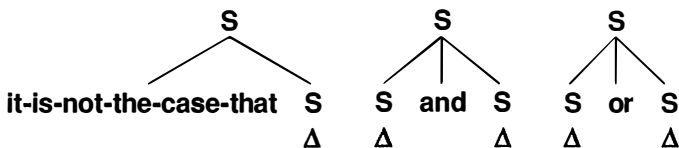
we wrote:

$\llbracket \text{Ann} \rrbracket = \text{Ann}.$

This lexical entry determines that the denotation of the English name “Ann” is the person Ann. The distinction between expressions and their denotations is important, so we will usually use *some* notational device to indicate the difference. We will never write things like $\llbracket \llbracket \text{Ann} \rrbracket \rrbracket$. This would have to be read as “the denotation of (the person) Ann”, and thus is nonsense. And we will also avoid using bold-face for purposes other than to replace quotes (such as emphasis, for which we use italics).

Exercise on sentence connectives

Suppose we extend our fragment to include phrase structures of the forms below (where the embedded S-constituents may either belong to the initial fragment or have one of these three forms themselves):



How do we have to revise and extend the semantic component in order to provide all the phrase structures in this expanded fragment with interpretations? Your task in this exercise is to define an appropriate semantic value for each new lexical item (treat “it-is-not-the-case-that” as a single lexical item here) and to write appropriate semantic rules for the new types of non-terminal nodes. To do this, you will also have to expand the inventory of possible semantic values. Make sure that you stick to our working hypothesis that all semantic composition is functional application (Frege’s Conjecture).

2.2 Sets and their characteristic functions⁵

We have construed the denotations of intransitive verbs as functions from individuals to truth-values. Alternatively, they are often regarded as *sets* of individuals. This is the standard choice for the extensions of 1-place predicates in logic. The intuition here is that each verb denotes the set of those things that it is true of. For example: $\llbracket \text{sleeps} \rrbracket = \{x \in D : x \text{ sleeps}\}$.⁶ This type of denotation would require a different semantic rule for composing subject and predicate, one that isn't simply functional application.

Exercise

Write the rule it would require.

Here we have chosen to take Frege's Conjecture quite literally, and have avoided sets of individuals as denotations for intransitive verbs. But for some purposes, sets are easier to manipulate intuitively, and it is therefore useful to be able to pretend in informal talk that intransitive verbs denote sets. Fortunately, this make-believe is harmless, because there exists a one-to-one correspondence between sets and certain functions.

- (1) Let A be a set. Then char_A , the *characteristic function* of A , is that function f such that, for any $x \in A$, $f(x) = 1$, and for any $x \notin A$, $f(x) = 0$.
- (2) Let f be a function with range $\{0, 1\}$. Then char_f , the set characterized by f , is $\{x \in D : f(x) = 1\}$.

Exploiting the correspondence between sets and their characteristic functions, we will often switch back and forth between function talk and set talk in the discussion below, sometimes saying things that are literally false, but become true when the references to sets are replaced by references to their characteristic functions (or vice versa).

Here is an illustration: Suppose our universe consists of three individuals, $D = \{\text{Ann}, \text{Jan}, \text{Maria}\}$. Suppose further that Ann and Jan are the ones who sleep, and Ann is the only one who snores. If we treat intransitive verbs as denoting sets, we may then assign the following denotations to *sleep* and *snores*:

- (3) $\llbracket \text{sleep} \rrbracket = \{\text{Ann}, \text{Jan}\}$.
- (4) $\llbracket \text{snores} \rrbracket = \{\text{Ann}\}$.

We can now write things like the following:

$$(5) \text{ Ann} \in \llbracket \text{sleep} \rrbracket.$$

$$(6) \llbracket \text{snore} \rrbracket \subseteq \llbracket \text{sleep} \rrbracket.$$

$$(7) |\llbracket \text{snore} \rrbracket \cap \llbracket \text{sleep} \rrbracket| = 1.$$

$|A|$ (the *cardinality* of A) is the number of elements in the set A .

(5) means that Ann is among the sleepers, (6) means that the snorers are a subset of the sleepers, and (7) means that the intersection of the snorers and the sleepers has exactly one element. All these are true, given (3) and (4). Now suppose we want to switch to a treatment under which intransitive verbs denote characteristic functions instead of the corresponding sets.

$$(3') \llbracket \text{sleep} \rrbracket = \begin{bmatrix} \text{Ann} \rightarrow 1 \\ \text{Jan} \rightarrow 1 \\ \text{Maria} \rightarrow 0 \end{bmatrix}$$

$$(4') \llbracket \text{snore} \rrbracket = \begin{bmatrix} \text{Ann} \rightarrow 1 \\ \text{Jan} \rightarrow 0 \\ \text{Maria} \rightarrow 0 \end{bmatrix}$$

If we want to make statements with the same import as (5)–(7) above, we can no longer use the same formulations. For instance, the statement

$$\text{Ann} \in \llbracket \text{sleep} \rrbracket$$

if we read it literally, is now false. According to (3'), $\llbracket \text{sleep} \rrbracket$ is a function. Functions are sets of ordered pairs, in particular,

$$\llbracket \text{sleep} \rrbracket = \{ \langle \text{Ann}, 1 \rangle, \langle \text{Jan}, 1 \rangle, \langle \text{Maria}, 0 \rangle \}.$$

Ann, who is not an ordered pair, is clearly not among the elements of this set. Likewise,

$$\llbracket \text{snore} \rrbracket \subseteq \llbracket \text{sleep} \rrbracket$$

is now false, because there is one element of $\llbracket \text{snore} \rrbracket$, namely the pair $\langle \text{Jan}, 0 \rangle$, which is not an element of $\llbracket \text{sleep} \rrbracket$. And

$$|\llbracket \text{snore} \rrbracket \cap \llbracket \text{sleep} \rrbracket| = 1$$

is false as well, because the intersection of the two functions described in (3') and (4') contains not just one element, but two, namely $\langle \text{Ann}, 1 \rangle$ and $\langle \text{Maria}, 0 \rangle$.

The upshot of all this is that, once we adopt (3') and (4') instead of (3) and (4), we have to express ourselves differently if we want to make statements that preserve the intuitive meaning of our original (5)–(7). Here is what we have to write instead:

$$(5') \quad \llbracket \text{sleep} \rrbracket(\text{Ann}) = 1$$

$$(6') \quad \text{For all } x \in D : \text{ if } \llbracket \text{snore} \rrbracket(x) = 1, \text{ then } \llbracket \text{sleep} \rrbracket(x) = 1$$

Or, equivalently:

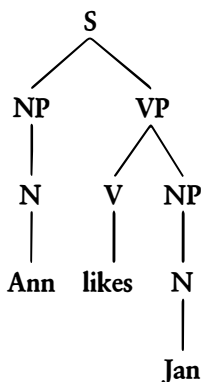
$$\{x : \llbracket \text{snore} \rrbracket(x) = 1\} \subseteq \{x : \llbracket \text{sleep} \rrbracket(x) = 1\}$$

$$(7') \quad |\{x : \llbracket \text{snore} \rrbracket(x) = 1\} \cap \{x : \llbracket \text{sleep} \rrbracket(x) = 1\}| = 1$$

As you can see from this, using characteristic functions instead of sets makes certain things a little more cumbersome.

2.3 Adding transitive verbs: semantic types and denotation domains

Our next goal is to extend our semantics to simple transitive clauses like “Ann likes Jan”. We take it that the structures that the syntax assigns to these look like this:



The transitive verb combines with its direct object to form a VP, and the VP combines with the subject to form a sentence. Given structures of this kind, what is a suitable denotation for transitive verbs? Look at the above tree. The lexical item “likes” is dominated by a non-branching V-node. The denotation of “likes”, then, is passed up to this node. The next node up is a branching VP-node. Assuming that semantic interpretation is local, the denotation of this VP-node must be composed from the denotations of its two daughter nodes. If Frege’s Conjecture is right, this composition process amounts to functional application. We have seen before that the denotations of NP-nodes dominating proper names are individuals. And that the denotation of VP-nodes are functions from individuals to truth-values. This means that the denotation of a transitive verb like “likes” is a *function from individuals to functions from individuals to truth-values*.⁷

When we define such a function-valued function in full explicitness, we have to nest one definition of a function inside another. Here is the proposed meaning of “likes”.

$$\begin{aligned} \llbracket \text{like} \rrbracket &= f : D \rightarrow \{g : g \text{ is a function from } D \text{ to } \{0, 1\}\} \\ \text{For all } x \in D, f(x) &= g_x : D \rightarrow \{0, 1\} \\ \text{For all } y \in D, g_x(y) &= 1 \text{ iff } y \text{ likes } x. \end{aligned}$$

This reads: $\llbracket \text{like} \rrbracket$ is that function f from D into the set of functions from D to $\{0, 1\}$ such that, for all $x \in D$, $f(x)$ is that function g_x from D into $\{0, 1\}$ such that, for all $y \in D$, $g_x(y) = 1$ iff y likes x . Fortunately, this definition can be compressed a bit. There is no information lost in the following reformulation:⁸

$$\begin{aligned} \llbracket \text{like} \rrbracket &= f : D \rightarrow \{g : g \text{ is a function from } D \text{ to } \{0, 1\}\} \\ \text{For all } x, y \in D, f(x)(y) &= 1 \text{ iff } y \text{ likes } x. \end{aligned}$$

$\llbracket \text{like} \rrbracket$ is a 1-place function, that is, a function with just one argument. The arguments of $\llbracket \text{like} \rrbracket$ are interpreted as the individuals which are liked; that is, they correspond to the grammatical *object* of the verb “like”. This is so because we have assumed that transitive verbs form a VP with their direct object. The direct object, then, is the argument that is closest to a transitive verb, and is therefore semantically processed before the subject.

What is left to spell out are the interpretations for the new kinds of non-terminal nodes:

$$(S6) \quad \text{If } \alpha \text{ has the form } \begin{array}{c} \text{VP} \\ \swarrow \searrow \\ \beta \quad \gamma \end{array}, \text{ then } \llbracket \alpha \rrbracket = \llbracket \beta \rrbracket(\llbracket \gamma \rrbracket).$$

What we just proposed implies an addition to our inventory of possible denotations: aside from individuals, truth-values, and functions from individuals to truth-values, we now also employ functions from individuals to functions from individuals to truth-values. It is convenient at this point to introduce a way of systematizing and labeling the types of denotations in this growing inventory. Following a common practice in the tradition of Montague, we employ the labels “e” and “t” for the two basic types.⁹

- (1) e is the type of individuals.

$$D_e := D.$$

- (2) t is the type of truth-values.

$$D_t := \{0, 1\}.$$

Generally, D_τ is the set of possible denotations of type τ . Besides the basic types e and t, which correspond to Frege’s *saturated* denotations, there are derived types for various sorts of functions, Frege’s *unsaturated* denotations. These are labeled by ordered pairs of simpler types: $\langle\sigma, \tau\rangle$ is defined as the type of functions whose arguments are of type σ and whose values are of type τ . The particular derived types of denotations that we have employed so far are $\langle e, t \rangle$ and $\langle e, \langle e, t \rangle \rangle$:

- (3) $D_{\langle e, t \rangle} := \{f : f \text{ is a function from } D_e \text{ to } D_t\}$

- (4) $D_{\langle e, \langle e, t \rangle \rangle} := \{f : f \text{ is a function from } D_e \text{ to } D_{\langle e, t \rangle}\}$

Further additions to our type inventory will soon become necessary. Here is a general definition:

- (5) **Semantic types**

- (a) e and t are semantic types.
- (b) If σ and τ are semantic types, then $\langle\sigma, \tau\rangle$ is a semantic type.
- (c) Nothing else is a semantic type.

Semantic denotation domains

- (a) $D_e := D$ (the set of individuals).
- (b) $D_t := \{0, 1\}$ (the set of truth-values).
- (c) For any semantic types σ and τ , $D_{\langle\sigma, \tau\rangle}$ is the set of all functions from D_σ to D_τ .

(5) presents a recursive definition of an infinite set of semantic types and a parallel definition of a typed system of denotation domains. Which semantic

types are actually used by natural languages is still a matter of debate. The issue of “type economy” will pop up at various places throughout this book, most notably in connection with adjectives in chapter 4 and quantifier phrases in chapter 7. So far, we have encountered denotations of types e , $\langle e, t \rangle$, and $\langle e, \langle e, t \rangle \rangle$ as possible denotations for lexical items: D_e contains the denotations of proper names, $D_{\langle e, t \rangle}$ the denotations of intransitive verbs, and $D_{\langle e, \langle e, t \rangle \rangle}$ the denotations of transitive verbs. Among the denotations of non-terminal constituents, we have seen examples of four types: D_t contains the denotations of all Ss, $D_e (= D)$ the denotations of all Ns and NPs, $D_{\langle e, t \rangle}$ the denotations of all VPs and certain Vs, and $D_{\langle e, \langle e, t \rangle \rangle}$ the denotations of the remaining Vs.

2.4 Schönfinkelization

Once more we interrupt the construction of our semantic component in order to clarify some of the underlying mathematics. Our current framework implies that the denotations of transitive verbs are 1-place functions. This follows from three assumptions about the syntax–semantics interface that we have been making:

Binary Branching

In the syntax, transitive verbs combine with the direct object to form a VP, and VPs combine with the subject to form a sentence.

Locality

Semantic interpretation rules are local: the denotation of any non-terminal node is computed from the denotations of its daughter nodes.

Frege’s Conjecture

Semantic composition is functional application.

If transitive verbs denote 1-place function-valued functions, then our semantics contrasts with the standard semantics for 2-place predicates in logic, and it is instructive to reflect somewhat systematically on how the two approaches relate to each other.

In logic texts, the extension of a 2-place predicate is usually a set of ordered pairs: that is, a relation in the mathematical sense. Suppose our domain D contains just the three goats Sebastian, Dimitri, and Leopold, and among these, Sebastian is the biggest and Leopold the smallest. The relation “is-bigger-than” is then the following set of ordered pairs:

$R_{\text{bigger}} = \{ \langle \text{Sebastian}, \text{Dimitri} \rangle, \langle \text{Sebastian}, \text{Leopold} \rangle, \langle \text{Dimitri}, \text{Leopold} \rangle \}$.

We have seen above that there is a one-to-one correspondence between sets and their characteristic functions. The “functional version” of R_{bigger} is the following function from $D \times D$ to $\{0, 1\}$.¹⁰

$$f_{\text{bigger}} = \begin{bmatrix} \langle L, S \rangle \rightarrow 0 \\ \langle L, D \rangle \rightarrow 0 \\ \langle L, L \rangle \rightarrow 0 \\ \langle S, L \rangle \rightarrow 1 \\ \langle S, D \rangle \rightarrow 1 \\ \langle S, S \rangle \rightarrow 0 \\ \langle D, L \rangle \rightarrow 1 \\ \langle D, S \rangle \rightarrow 0 \\ \langle D, D \rangle \rightarrow 0 \end{bmatrix}$$

f_{bigger} is a 2-place function.¹¹ In his paper “Über die Bausteine der mathematischen Logik,”¹² Moses Schönfinkel showed how n -place functions can quite generally be reduced to 1-place functions. Let us apply his method to the 2-place function above. That is, let us *Schönfinkel*¹³ the function f_{bigger} . There are two possibilities. f'_{bigger} is the left-to-right Schönfinkelization of f_{bigger} . f''_{bigger} is the right-to-left Schönfinkelization of f_{bigger} .

$$f'_{\text{bigger}} = \begin{bmatrix} L \rightarrow \begin{bmatrix} L \rightarrow 0 \\ S \rightarrow 0 \\ D \rightarrow 0 \end{bmatrix} \\ S \rightarrow \begin{bmatrix} L \rightarrow 1 \\ S \rightarrow 0 \\ D \rightarrow 1 \end{bmatrix} \\ D \rightarrow \begin{bmatrix} L \rightarrow 1 \\ S \rightarrow 0 \\ D \rightarrow 0 \end{bmatrix} \end{bmatrix}$$

f'_{bigger} is a function that applies to the first argument of the “bigger” relation to yield a function that applies to the second argument. When applied to Leopold, it yields a function that maps any goat into 1 if it is smaller than Leopold. There is no such goat. Hence we get a constant function that assigns 0 to all the goats. When applied to Sebastian, f'_{bigger} yields a function that maps any goat into 1 if it is smaller than Sebastian. There are two such goats, Leopold and Dimitri. And when applied to Dimitri, f'_{bigger} yields a function that maps any goat into 1 if it is smaller than Dimitri. There is only one such goat, Leopold.

$$f''_{\text{bigger}} = \left[\begin{array}{l} L \rightarrow \left[\begin{array}{l} L \rightarrow 0 \\ S \rightarrow 1 \\ D \rightarrow 1 \end{array} \right] \\ S \rightarrow \left[\begin{array}{l} L \rightarrow 0 \\ S \rightarrow 0 \\ D \rightarrow 0 \end{array} \right] \\ D \rightarrow \left[\begin{array}{l} L \rightarrow 0 \\ S \rightarrow 1 \\ D \rightarrow 0 \end{array} \right] \end{array} \right]$$

f''_{bigger} is a function that applies to the second argument of the “bigger” relation to yield a function that applies to the first argument. When applied to Leopold, it yields a function that maps any goat into 1 if it is bigger than Leopold. These are all the goats except Leopold. When applied to Sebastian, f''_{bigger} yields a function that maps any goat into 1 if it is bigger than Sebastian. There is no such goat. And when applied to Dimitri, f''_{bigger} maps any goat into 1 if it is bigger than Dimitri. There is only one such goat, Sebastian.

On both methods, we end up with nothing but 1-place functions, and this is as desired. This procedure can be generalized to any n -place function. You will get a taste for this by doing the exercises below.

Now we can say how the denotations of 2-place predicates construed as relations are related to the Fregean denotations introduced above. The Fregean denotation of a 2-place predicate is the right-to-left Schönfinkelled version of the characteristic function of the corresponding relation. Why the *right-to-left* Schönfinkelization? Because the corresponding relations are customarily specified in such a way that the grammatical object argument of a predicate corresponds to the *right* component of each pair in the relation, and the subject to the left one. (For instance, by the “love”-relation one ordinarily means the set of lover–loved pairs, in this order, and not the set of loved–lover pairs.) That’s an arbitrary convention, in a way, though suggested by the linear order in which English realizes subjects and objects. As for the Fregean denotations of 2-place predicates, remember that it is not arbitrary that their (unique) argument corresponds to the grammatical object of the predicate. Since the object is closest to the predicate in hierarchical terms, it must provide the argument for the function denoted by the predicate.

Exercise 1

Suppose that our universe D contains just two elements, Jacob and Maria. Consider now the following binary and ternary relations:

$$R_{\text{adores}} = \{ \langle \text{Jacob, Maria} \rangle, \langle \text{Maria, Maria} \rangle \}$$

$$R_{\text{assigns to}} = \{ \langle \text{Jacob, Jacob, Maria} \rangle, \langle \text{Maria, Jacob, Maria} \rangle \}$$

In standard predicate logic, these would be suitable extensions for the 2-place and 3-place predicate letters “F²” and “G³” as used in the following scheme of abbreviation:

“F²”: “a adores b”

“G³”: “a assigns b to c”

Find the characteristic functions for both of these relations, and then Schönfinkel them from right to left. Could the two Schönfinkelled functions be suitable denotations for the English verbs “adore” and “assign (to)” respectively? If yes, why? If not, why not?

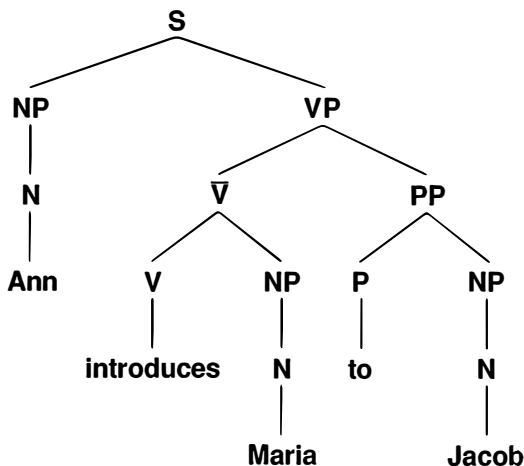
Exercise 2

In the exercise on sentence connectives in section 2.1, we stipulated ternary branching structures for sentences with “and” and “or”. Now assume that all English phrase structures are at most binary branching, and assign accordingly revised syntactic analyses to these sentences. (Whether you choose right-branching or left-branching structures does not matter here, but stick to one option.) Then revise the semantics accordingly. As always, be sure to provide every subtree with a semantic value, as well as to adhere to our current assumptions about the semantic interpretation component (Locality and Frege’s Conjecture).

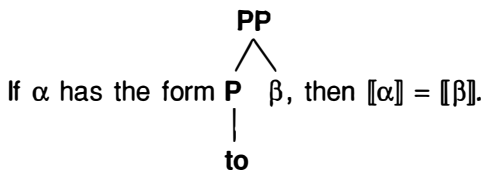
Using the labeling system introduced at the end of section 2.3, specify the type of denotation for each node in your binary branching structure for the sentence “Jan works, and it is not the case that Jan smokes”.

Exercise 3

(a) Extend the fragment in such a way that phrase structure trees of the following kind are included.



Add the necessary semantic rules and lexical entries, sticking to Locality and Frege's Conjecture. Assume that the preposition "to" is a semantically vacuous element; that is, assume the *ad hoc* rule below:



(b) Suppose now that the actual world contains just three individuals, Ann, Maria, and Jacob. And suppose further that Ann introduces Maria to Jacob, and Maria introduces Jacob to Ann, and no further introductions take place. Which particular function is $[[\text{introduce}]]$ in this case? Display it in a *table*.

(c) Using the *table* specification of $[[\text{introduce}]]$ from (b) and the lexical entries for the names, calculate the denotations of each non-terminal node in the tree under (a).

(d) Under standard assumptions, a predicate logic formalization of the English sentence "Ann introduces Maria to Jacob" might look like this:

$I^3(A\ M\ J)$

Scheme of abbreviation:

"A": "Ann"

"M": "Maria"

"J": "Jacob"

" I^3 ": "a introduces b to c"

The extension of “I³” under this scheme of abbreviation is the following set X of ordered triples:

$$X := \{ \langle x, y, z \rangle \in D \times D \times D : x \text{ introduces } y \text{ to } z \}.$$

How is this extension related to the extension – let’s call it *f* – for **introduce** that you defined in (a)? Give your answer by completing the following statement:

$$\text{For any } x, y, z \in D, f(x)(y)(z) = 1 \text{ iff } \dots \in X.$$

2.5 Defining functions in the λ -notation

The final section of this chapter is devoted to yet another technical matter. You have already had a taste of the ubiquity of functions among the denotations that our Fregean semantics assigns to the words and phrases of natural languages. We will now introduce another notation for describing functions, which will save us some ink in future chapters.

The format in which we have defined most of our functions so far was introduced in section 1.3 with the following example:

$$(1) \quad F_{+1} = f : \mathbb{N} \rightarrow \mathbb{N}$$

For every $x \in \mathbb{N}$, $f(x) = x + 1$.

The same definition may henceforth be expressed as follows:

$$(2) \quad F_{+1} := [\lambda x : x \in \mathbb{N} . x + 1]$$

The λ -term, “ $\lambda x : x \in \mathbb{N} . x + 1$ ”, is to be read as “the (smallest) function which maps every x such that x is in $\mathbb{N}$ to $x + 1$ ”.

Generally, λ -terms are constructed according to the following schema:

$$(3) \quad [\lambda \alpha : \phi . \gamma]$$

We say that α is the *argument variable*, ϕ the *domain condition*, and γ the *value description*. The domain condition is introduced by a colon, and the value description by a period.¹⁴ α will always be a letter standing for an arbitrary argument of the function we are trying to define. In (2), this is the letter “ x ”, which we generally use to stand for arbitrary individuals. The domain condition ϕ defines the domain of our function, and it does this by placing a condition on

possible values for α . In our example, ϕ corresponds to " $x \in \mathbb{N}$ ", which encodes the information that the domain of F_{+1} contains all and only the natural numbers. The value description γ , finally, specifies the value that our function assigns to the arbitrary argument represented by α . In (2), this reads " $x + 1$ ", which tells us that the value that F_{+1} assigns to each argument is that argument's successor. The general convention for reading λ -terms in (semi-mathematical) English is such that (3) reads as (3')

(3') the smallest function which maps every α such that ϕ to γ

We will typically omit "smallest", but it is always understood, and it is strictly speaking necessary to pick out the intended function uniquely. Notice, for instance, that besides F_{+1} , there are lots of other functions which also map every natural number to its successor: namely, all those functions which are supersets of F_{+1} , but have larger domains than $\mathbb{N}$. By adding "smallest", we make explicit that the domain condition ϕ delimits the domain exactly; that is, that in (2), for instance, " $x \in \mathbb{N}$ " is not only a sufficient, but also a necessary, condition for " $x \in \text{dom}(F_{+1})$ ".¹⁵

Like other function terms, λ -terms can be followed by argument terms. So we have:

(4) $[\lambda x : x \in \mathbb{N} . x + 1](5) = 5 + 1 = 6$.

The λ -notation as we have just introduced it is not as versatile as the format in (1). It cannot be used to abbreviate descriptions of functions which involve a distinction between two or more cases. To illustrate this limitation, let's look at the function G defined in (5).

(5) $G = f : \mathbb{N} \rightarrow \mathbb{N}$

For every $x \in \mathbb{N}$, $f(x) = 2$, if x is even, and $f(x) = 1$ otherwise.

The problem we encounter if we attempt to press this into the shape " $[\lambda\alpha : \phi . \gamma]$ " is that there is no suitable value description γ . Obviously, neither "1" nor "2" is the right choice. $[\lambda x : x \in \mathbb{N} . 1]$ would be that function which maps every natural number to 1, clearly a different function from the one described in (5). And similarly, of course, " $[\lambda x : x \in \mathbb{N} . 2]$ " would be inappropriate.¹⁶ This implies that, as it stands, the new notation is unsuitable for precisely the kinds of functions that figure most prominently in our semantics. Take the extension of an intransitive verb.

(6) $[\text{smoke}] = f : D \rightarrow \{0, 1\}$

For every $x \in D$, $f(x) = 1$ iff x smokes.

(6) stipulates that $f(x)$ is to be 1 if x smokes and 0 otherwise.¹⁷ So we face the same difficulty as with (5) above when it comes to deciding on the value description in a suitable λ -term.

We will get rid of this difficulty by defining an extended use of the λ -notation. So far, you have been instructed to read “ $[\lambda\alpha : \phi . \gamma]$ ” as “the function which maps every α such that ϕ to γ ”. This paraphrase makes sense only when the value description γ is a noun phrase. If γ had the form of a sentence, for instance, we would get little more than word salad. Try reading out a λ -term like “ $[\lambda x : x \in \mathbb{N} . x \text{ is even}]$ ”. What comes out is: “the function which maps every x in $\mathbb{N}$ to x is even”. This is neither colloquial English nor any kind of technical jargon; it just doesn’t make any sense.

We could, of course, change the instructions and stipulate a different way to read the notation. Suppose we decided that “ $[\lambda\alpha : \phi . \gamma]$ ” was to be read as follows: “the function which maps every α such that ϕ to 1 if γ and to 0 otherwise”. Then, of course, “ $[\lambda x : x \in \mathbb{N} . x \text{ is even}]$ ”, would make perfect sense: “the function which maps every x in $\mathbb{N}$ to 1, if x is even, and to 0 otherwise”. With this new convention in force, we could also use the λ -notation to abbreviate our lexical entry for “smoke”.

(7) $\llbracket \text{smoke} \rrbracket := [\lambda x : x \in D . x \text{ smokes}]$

If this reads: “let $\llbracket \text{smoke} \rrbracket$ be the function which maps every x in D to 1, if x smokes, and to 0 otherwise”, it is easily seen as an equivalent reformulation of (6). If we add an argument term, we have:

(8) $\llbracket \text{smoke} \rrbracket(\text{Ann}) = [\lambda x : x \in D . x \text{ smokes}](\text{Ann}) = 1$ if Ann smokes
 $= 0$ otherwise.

Instead of (8), we’ll write:

(8’) $\llbracket \text{smoke} \rrbracket(\text{Ann}) = [\lambda x : x \in D . x \text{ smokes}](\text{Ann}) = 1$ iff Ann smokes.

The down side of substituting this new convention for reading λ -terms for the previous one would be that it makes garbage of those cases which we considered at first. For instance, we could no longer write things like “ $[\lambda x : x \in \mathbb{N} . x + 1]$ ” if this had to be read: “the function which maps every x in $\mathbb{N}$ to 1, if $x + 1$ [sic], and to 0 otherwise”.

We will have our cake and eat it too, by stipulating that λ -terms may be read in either one of the two ways which we have considered, whichever happens to make sense in the case at hand.

- (9) Read “[$\lambda\alpha : \phi . \gamma$]” as either (i) or (ii), whichever makes sense.
- (i) “the function which maps every α such that ϕ to γ ”
 - (ii) “the function which maps every α such that ϕ to 1, if γ , and to 0 otherwise”

Luckily, this convention doesn’t create any ambiguity, because only one clause will apply in each given case. If γ is a sentence, that’s clause (ii), otherwise (i).¹⁸

You may wonder why the same notation has come to be used in two distinct senses. Wouldn’t it have been wiser to have two different notations? This is a legitimate criticism. There does exist a precise *and uniform* interpretation of the λ -notation, where λ -terms can be shown to have the same meaning in the two cases, after all, despite the superficial disparity of the English paraphrases. This would require a formalization of our current informal metalanguage, however. We would map phrase structure trees into expressions of a λ -calculus, which would in turn be submitted to semantic interpretation.¹⁹ Here, we use λ -operators and variables informally in the metalanguage, and rely on a purely intuitive grasp of the technical locutions and notations involving them (as is the practice, by the way, in most mathematical and scientific texts). Our use of the λ -notation in the metalanguage, then, has the same status as our informal use of other technical notation, the abstraction notation for sets, for example.

The relative conciseness of the λ -notation makes it especially handy for the description of function-valued functions. Here is how we can express the lexical entry of a transitive verb.

- (10) $\llbracket \text{love} \rrbracket := [\lambda x : x \in D . [\lambda y : y \in D . y \text{ loves } x]]$

In (10), we have a big λ -term, whose value description is a smaller λ -term. Which clause of the reading convention in (9) applies to each of these? In the smaller one (“ $[\lambda y : y \in D . y \text{ loves } x]$ ”), the value description is evidently sentential, so we must use clause (ii) and read this as “the function which maps every y in D to 1, if y loves x , and to 0 otherwise”. And since this phrase is a noun phrase, clause (i) must apply for the bigger λ -term, and thus that one reads: “the function which maps every x in D to the function which maps every y in D to 1, if y loves x , and to 0 otherwise”.

Functions can have functions as arguments. Here, too, the λ -notation is handy. Take:

- (11) $[\lambda f : f \in D_{\langle e, t \rangle} . \text{there is some } x \in D_e \text{ such that } f(x) = 1]$

The function in (11) maps functions from $D_{\langle e, t \rangle}$ into truth-values. Its arguments, then, are functions from individuals to truth-values. The function in (12) is a possible argument:

$$(12) \quad [\lambda y : y \in D_e . y \text{ stinks}]$$

If we apply the function in (11) to the function in (12), we get the value 1 if there is some $x \in D_e$ such that $[\lambda y : y \in D_e . y \text{ stinks}](x) = 1$. Otherwise, the value is 0. That is, we have:

$$(13) \quad [\lambda f : f \in D_{\langle e, t \rangle} . \text{there is some } x \in D_e \text{ such that } f(x) = 1]([\lambda y : y \in D_e . y \text{ stinks}]) = 1$$

iff there is some $x \in D_e$ such that $[\lambda y : y \in D_e . y \text{ stinks}](x) = 1$
 iff there is some $x \in D_e$ such that $x \text{ stinks}$.

Let us introduce a couple of abbreviatory conventions which will allow us to describe the most common types of functions we will be using in this book even more concisely. First, we will sometimes omit the outermost brackets around a λ -term which is not embedded in a larger formal expression. Second, we will contract the domain condition when it happens to be of the form “ $\alpha \in \beta$ ”. Instead of “ $\lambda\alpha : \alpha \in \beta . \gamma$ ”, we will then write “ $\lambda\alpha \in \beta . \gamma$ ”. (This corresponds to shortening the paraphrase “every α such that α is in β ” to “every α in β ”.) And sometimes we will leave out the domain condition altogether, notably when it happens to be “ $x \in D$ ”. So the lexical entry for “love” may appear, for example, in either of the following shorter versions:

$$(14) \quad \begin{aligned} \llbracket \text{love} \rrbracket &:= \lambda x \in D . [\lambda y \in D . y \text{ loves } x] \\ \llbracket \text{love} \rrbracket &:= \lambda x . [\lambda y . y \text{ loves } x] \end{aligned}$$

You have to be careful when λ -terms are followed by argument terms. Without further conventions, 15(a) is not legitimate, since it is ambiguous between 15(b) and 15(c), which are not equivalent:

$$(15) \quad \begin{aligned} (a) \quad &\lambda x \in D . [\lambda y \in D . y \text{ loves } x](\text{Sue}) \\ (b) \quad &[\lambda x \in D . [\lambda y \in D . y \text{ loves } x](\text{Sue})] = \lambda x \in D . \text{Sue loves } x \\ (c) \quad &[\lambda x \in D . [\lambda y \in D . y \text{ loves } x]](\text{Sue}) = \lambda y \in D . y \text{ loves Sue} \end{aligned}$$

In cases of this kind, use either the formulation in (b) or the one in (c), whichever corresponds to the intended meaning.

There is a close connection between the abstraction notation for sets and the λ -notation for functions. The characteristic function of the set $\{x \in \mathbb{N} : x \neq 0\}$ is $[\lambda x \in \mathbb{N} . x \neq 0]$, for example. Much of what you have learned about the abstraction notation for sets in chapter 1 can now be carried over to the λ -notation for functions. Set talk can be easily translated into function talk. Here are the correspondences for some of the cases we looked at earlier:

Set talk

$29 \in \{x \in \mathbb{N} : x \neq 0\}$ iff $29 \neq 0$

$\text{Massachusetts} \in \{x \in D : \text{California is a western state}\}$ iff California is a western state.

$\{x \in D : \text{California is a western state}\} = D$ if California is a western state.

$\{x \in D : \text{California is a western state}\} = \emptyset$ if California is not a western state.

$\{x \in \mathbb{N} : x \neq 0\} = \{y \in \mathbb{N} : y \neq 0\}$

$\{x \in \mathbb{N} : x \in \{x \in \mathbb{N} : x \neq 0\}\} = \{x \in \mathbb{N} : x \neq 0\}$

$\{x \in \mathbb{N} : x \in \{y \in \mathbb{N} : y \neq 0\}\} = \{x \in \mathbb{N} : x \neq 0\}$

Function talk

$[\lambda x \in \mathbb{N} . x \neq 0](29) = 1$ iff $29 \neq 0$

$[\lambda x \in D . \text{California is a western state}](\text{Massachusetts}) = 1$ iff California is a western state.

$[\lambda x \in D . \text{California is a western state}](x) = 1$ for all $x \in D$ if California is a western state.

$[\lambda x \in D . \text{California is a western state}](x) = 0$ for all $x \in D$ if California is not a western state.

$[\lambda x \in \mathbb{N} : x \neq 0] = [\lambda y \in \mathbb{N} : y \neq 0]$

$[\lambda x \in \mathbb{N} . [\lambda x \in \mathbb{N} . x \neq 0](x)] = [\lambda x \in \mathbb{N} . x \neq 0]$

$[\lambda x \in \mathbb{N} . [\lambda y \in \mathbb{N} . y \neq 0](x)] = [\lambda x \in \mathbb{N} . x \neq 0]$

If you are still unclear about some of the statements in the left column, go back to chapter 1 and consult the questions and answers about the abstraction notation for sets. Once you understand the set talk in the left column, the transition to the function talk in the right column should be straightforward.

Exercise 1

Describe the following functions in words:

- (a) $\lambda x \in \mathbb{N} . x > 3$ and $x < 7$
- (b) $\lambda x : x$ is a person . x 's father
- (c) $\lambda X \in \text{Pow}(D) . \{y \in D : y \notin X\}$ ²⁰
- (d) $\lambda X : X \subseteq D . [\lambda y \in D . y \notin X]$

Exercise 2

In this exercise, simple functions are described in a rather complicated way. Simplify the descriptions as much as possible.

- (a) $[\lambda x \in D . [\lambda y \in D . [\lambda z \in D . z \text{ introduced } x \text{ to } y]]](\text{Ann})(\text{Sue})$
 - (b) $[\lambda x \in D . [\lambda y \in D . [\lambda z \in D . z \text{ introduced } x \text{ to } y](\text{Ann})](\text{Sue})$
 - (c) $[\lambda x \in D . [\lambda y \in D . [\lambda z \in D . z \text{ introduced } x \text{ to } y](\text{Ann})]](\text{Sue})$
 - (d) $[\lambda x \in D . [\lambda y \in D . [\lambda z \in D . z \text{ introduced } x \text{ to } y]](\text{Ann})](\text{Sue})$
 - (e) $[\lambda f \in D_{\langle e, t \rangle} . [\lambda x \in D_e . f(x) = 1 \text{ and } x \text{ is gray}]]([\lambda y \in D_e . y \text{ is a cat}])$
 - (f) $[\lambda f \in D_{\langle e, \langle e, t \rangle \rangle} . [\lambda x \in D_e . f(x)(\text{Ann}) = 1]]([\lambda y \in D_e . [\lambda z \in D_e . z \text{ saw } y]])$
 - (g) $[\lambda x \in |N . [\lambda y \in |N . y > 3 \text{ and } y < 7](x)]$
 - (h) $[\lambda z \in |N . [\lambda y \in |N . [\lambda x \in |N . x > 3 \text{ and } x < 7](y)](z)]$
-

Exercise 3

Suppose “and” and “or” have Schönfinkeled denotations, that is $[\text{and}]$ and $[\text{or}]$ are both members of $D_{\langle t, \langle t, t \rangle \rangle}$. They are functions that map truth-values into functions from truth-values to truth-values. Specify the two functions using the λ -notation.

Exercise 4

Replace the “?” in each of the following statements (you may want to review definition (5) of section 2.3 before tackling this exercise):

- (a) $[\lambda f \in D_{\langle e, t \rangle} . [\lambda x \in D_e . f(x) = 1 \text{ and } x \text{ is gray}]] \in D_?$
 - (b) $[\lambda f \in D_{\langle e, \langle e, t \rangle \rangle} . [\lambda x \in D_e . f(x)(\text{Ann}) = 1]] \in D_?$
 - (c) $[\lambda y \in D_e . [\lambda f \in D_{\langle e, t \rangle} . [\lambda x \in D_e . f(x) = 1 \text{ and } x \text{ is in } y]]] \in D_?$
 - (d) $[\lambda f \in D_{\langle e, t \rangle} . \text{there is some } x \in D_e \text{ such that } f(x) = 1] \in D_?$
 - (e) $[\lambda f \in D_{\langle e, t \rangle} . \text{Mary}] \in D_?$
 - (f) $[\lambda f \in D_{\langle e, t \rangle} . [\lambda g \in D_{\langle e, t \rangle} . \text{there is no } x \in D_e \text{ such that } f(x) = 1 \text{ and } g(x) = 1]] \in D_?$
-

Notes

- 1 Here and below, when we speak of the denotation of a *node* in a tree, we really mean the denotation of the subtree dominated by that node.
- 2 M. Black and P. Geach, *Translations from the Philosophical Writings of Gottlob Frege* (Oxford, Basil Blackwell, 1960), use “reference” to translate Frege’s “Bedeutung”.

Some translations, for instance, in A. P. Martinich (ed.), *The Philosophy of Language*, 2nd edn (New York and Oxford, Oxford University Press, 1990), and J. L. Garfield and M. Kiteley (eds), *The Essential Readings in Modern Semantics* (New York, Paragon House, 1991), use “nominatum”. This is the translation Rudolf Carnap introduced in *Meaning and Necessity. A Study in Semantics and Modal Logic* (Chicago, University of Chicago Press, 1947).

- 3 M. Dummett, *Frege. Philosophy of Language*, 2nd edn (Cambridge, Mass., Harvard University Press, 1981), p. 227.
- 4 The use of Wittgenstein’s term “show” in this connection is due to Dummett, *ibid.*
- 5 This is another mathematical-background section, which the mathematically sophisticated need only skim.
- 6 The notation “ $\{x \in D : x \text{ sleeps}\}$ ” is a standard abbreviation for “ $\{x : x \in D \text{ and } x \text{ sleeps}\}$ ”. (Recall that when we first introduced the set-abstraction notation, we allowed only a variable to the left of the colon.)
- 7 This conclusion, even though we are motivating it by using his general proposal about semantic composition, Frege himself would not have endorsed. As discussed by Dummett (*Frege*, pp. 40ff.), he did not allow for function-valued functions.
- 8 Notice that the implicit bracketing in “ $f(x)(y)$ ” is “ $[f(x)](y)$ ”, not “ $f[(x)(y)]$ ”. The latter wouldn’t make any sense. (What could we possibly mean by “ $(x)(y)$ ”?) So it is not necessary to make the correct parse explicit in the notation.
- 9 R. Montague, *Formal Philosophy* (New Haven, Yale University Press, 1974); “e” is for “entity”, “t” for “truth-value”.
- 10 $D \times D$ is the *Cartesian product* of D with D , which is defined as the set of ordered pairs of elements of D .
- 11 A 2-place function on a domain A is a function with domain $A \times A$, which is defined as $\{ \langle x, y \rangle : x \in A \text{ and } y \in A \}$.
- 12 *Mathematische Annalen*, 92 (1924), pp. 305–16.
- 13 This procedure is also called “Currying” after the logician H. B. Curry, who built on Schönfinkel’s work. Molly Diesing informs us that by Stephen Jay Gould’s “Brontosaurus principle”, one could argue that we should use “Currying”: generality of use takes priority over temporal precedence. We are not sure how general the use of “Currying” is at this time, however, hence we don’t know whether the Brontosaurus Principle applies to this case. We’ll stick to temporal priority, then. W. Kneale and M. Kneale, *The Development of Logic* (Oxford, Clarendon Press, 1962), pp. 522f., credit Schönfinkel for Schönfinkelization.
- 14 Most versions of the λ -notation in the literature look a little different. What we are calling the “domain condition” is typically absent, and the intended domain is indicated instead by using argument variables that are assigned to fixed semantic types. The value description is frequently enclosed in brackets rather than introduced by a period. The terms “argument variable”, “domain condition”, and “value description” are also our own invention.
- 15 The attentive reader may have noticed that another piece of information seems to get lost in the reformulation from (1) to (2): viz. information about which set F_{+1} is *into*. Nothing in (2) corresponds to the part “ $\rightarrow \text{IN}$ ” in (1). Is this a problem? No. If you go back to our initial definition of “function” in section 1.3, you can see that the information supplied in (2) is entirely sufficient to define a unique function. It already follows from (2) that all values of F_{+1} are in IN . In other words, the format employed in (1) is actually redundant in this respect.
- 16 The best we can do, if we insist on using the λ -notation to define G , is to describe G as the union of two separate functions with smaller domains:

$G := \lambda x : x \in \mathbb{N} \ \& \ x \text{ is even} . 2 \cup \lambda x : x \in \mathbb{N} \ \& \ x \text{ is odd} . 1$

But this is not much of an *abbreviation* of (5). Remember, we were planning to save ink.

- 17 Incidentally, the indication that f is into $\{0, 1\}$ is not redundant in (6). Without this information, we could not conclude that $f(x) = 0$ when x doesn't smoke. From the condition " $f(x) = 1$ iff x smokes" by itself, we can infer only that $f(x) \neq 1$ in this case. If we want to render " $\rightarrow \{0, 1\}$ " as redundant here as " $\rightarrow \mathbb{N}$ " was in (1) and (5), we have to replace " $f(x) = 1$ iff x smokes" by something like the formulation in the text below: " $f(x) = 1$ if x smokes and 0 otherwise'.
- 18 Even with this disjunctive convention, by the way, the λ -notation remains unsuitable for abbreviating definitions like the one for G in (5). But we can live with this, because functions like G are not common in semantic applications.
- 19 A. Church, "A Formulation of a Simple Theory of Types," *Journal of Symbolic Logic*, 5 (1940), pp. 56–68. See also L. T. F. Gamut, *Logic, Language and Meaning*, vol. 2: *Intensional Logic and Logical Grammar* (Chicago, University of Chicago Press, 1991).
- 20 "Pow(D)" reads "the power set of D ". The power set of a set is the set of all its subsets. (More formally: for any set X , $\text{Pow}(X) := \{Y : Y \subseteq X\}$.)

3 Semantics and Syntax

In defining the semantic component for the small fragment of English we considered in the previous chapter, we followed the traditional method of writing a special semantic rule for each syntactic configuration that we encountered in our trees. There was a rule for *Ss* dominating an NP and a VP, a rule for VPs dominating a V and an NP, a rule for VPs dominating just a V, and so on.¹ But if Frege's Conjecture is right, and all non-trivial semantic composition is functional application, then we shouldn't need all these construction-specific interpretation rules. It should be sufficient to specify the denotations of the lexical items, and the rest should follow automatically.

We begin the present chapter by implementing this attractive suggestion. Then we proceed to some reflections about the place of the semantic component within the grammar, especially its relation to the syntactic component. Our Fregean approach gives us an interesting perspective on the relation between a verb and its arguments, which has been discussed from more syntactic points of view throughout the history of generative grammar.

3.1 Type-driven interpretation

The term "type-driven interpretation" was coined by Ewan Klein and Ivan Sag, whose paper "Type-Driven Translation" criticized the construction-specific interpretation method of classical Montague Grammar and proposed essentially the same revision that we present in this section.²

We continue to assume that the input for the semantic component is a set of phrase structure trees. But we no longer allow semantic rules for specific types of subtrees like the ones we wrote in chapter 2. Instead, we posit three very general principles:

(1) *Terminal Nodes* (TN)

If α is a terminal node, $[\alpha]$ is specified in the lexicon.

(2) *Non-Branching Nodes* (NN)

If α is a non-branching node, and β is its daughter node, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket$.

(3) *Functional Application* (FA)

If α is a branching node, $\{\beta, \gamma\}$ is the set of α 's daughters, and $\llbracket \beta \rrbracket$ is a function whose domain contains $\llbracket \gamma \rrbracket$, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket(\llbracket \gamma \rrbracket)$.

Notice that (3) does not specify the linear order of β and γ . Nevertheless, it applies in a unique way to each given binary branching tree. If α is of the form $[_s \text{ NP VP}]$, we have to apply it in such a way that the right-hand daughter corresponds to β and the left-hand daughter to γ . How do we know? Because this is the only way to satisfy the condition that $\llbracket \beta \rrbracket$ must be a function whose domain contains $\llbracket \gamma \rrbracket$. If α is of the form $[_{vp} \text{ V NP}]$, it's the other way round: β must be the left node, and γ the right one. Here you can see, by the way, what is behind the name "type-driven interpretation": it's the semantic types of the daughter nodes that determine the procedure for calculating the meaning of the mother node. The semantic interpretation component, then, can ignore certain features that syntactic phrase structure trees are usually assumed to have. All it has to see are the lexical items and the hierarchical structure in which they are arranged. Syntactic category labels and linear order are irrelevant.

During the course of the book, we will add one or two additional principles to the above list, but we will strive to keep it as parsimonious as possible. When we look at a new construction for which we don't yet have a semantic analysis, we always try first to accommodate it by adding only to the lexicon.

Our current set of lexical entries is just the beginning of a long list that we will extend as we go:

(4) Sample of lexical entries:

(i) $\llbracket \text{Ann} \rrbracket = \text{Ann}$

(ii) $\llbracket \text{smokes} \rrbracket = \lambda x \in D_e . x \text{ smokes}$

(iii) $\llbracket \text{loves} \rrbracket = \lambda x \in D_e . [\lambda y \in D_e . y \text{ loves } x]$

(iv) $\llbracket \text{and} \rrbracket = \begin{bmatrix} 1 \rightarrow \begin{bmatrix} 1 \rightarrow 1 \\ 0 \rightarrow 0 \end{bmatrix} \\ 0 \rightarrow \begin{bmatrix} 1 \rightarrow 0 \\ 0 \rightarrow 0 \end{bmatrix} \end{bmatrix}$

or, using the λ -notation:

$\llbracket \text{and} \rrbracket = \lambda p \in D_t . [\lambda q \in D_t . p = q = 1]^3$

etc.

Given the lexicon, the three interpretive principles TN, NN, and FA should suffice to derive all the predictions that were made by our old semantic component

from chapter 2. For the most part, it is transparent how this works out. The semantic rules (S2)–(S5) in chapter 2 are evidently special cases of NN, and (S1) and (S6) are special cases of FA. What is *not* covered by the new theory is the interpretation of ternary trees, like those we initially assumed for sentences with “and” and “or”. The semantic rules from the exercise on connectives in section 2.1 cannot be seen as special cases of any current principle. (Our formulation of FA above stipulates that α has no more than two daughters.) But rather than worry about this limitation, we will henceforth assume (mainly for pedagogical reasons) that phrase structures in natural language are at most binary branching.⁴ If necessary, it would not be too difficult to adjust our system of composition rules so that it could interpret a more realistic range of input structures.

3.2 The structure of the input to semantic interpretation

We have been assuming that the entities which are assigned denotations by the semantic component are phrase structure trees, and that these are somehow generated by the syntactic component of the grammar. We have not committed ourselves to any concrete assumptions about the kind of syntax that does this. Indeed, a variety of views on this matter are compatible with our approach to semantics and have been entertained. According to the so-called “Standard Theory” of early generative grammar, the input to semantic interpretation consisted of the Deep Structures generated by the base component of the syntax, essentially a context-free grammar.⁵ In Generative Semantics, Deep Structures were representations that resembled formulas of the Predicate Calculus, and could involve decomposition of predicates. From a more modern Chomskyan perspective, the inputs to semantic interpretation are Logical Forms, which are the output of transformational derivations. Many other syntactic theories can be, and have been, combined with the same kind of semantic theory – for instance, categorial grammars and monostratal phrase structure grammars.⁶ The only requirement for the syntax is that it provides us with phrase structure trees.

When we say “(phrase structure) trees”, what exactly do we mean? According to standard definitions, a phrase structure tree consists of a (finite) set of *labeled nodes* which are ordered by a *dominance* relation and a *linear precedence* relation.⁷ Many structural concepts can be defined in terms of dominance and precedence, among them the ones that happen to be referred to by our interpretation principles TN, NN, and FA: namely, “(non-) terminal node”, “(non-) branching”, and “daughter”. This being so, phrase structure trees in the standard sense are suitable structures for our semantic rules to apply to them.

We mentioned already that certain somewhat more impoverished structures would be suitable as well. Nothing in our semantic component – in particular, none of our principles TN, NN, and FA – makes any direct or indirect mention of non-terminal node labels or linear precedence. “(Non-) terminal node”, “(non-) branching”, and “daughter” are all definable in terms of the dominance relation alone, and these are the only syntactic notions that have been mentioned. We therefore conclude that our semantics can also be combined with a syntax that provides unlinearized structures without syntactic category labels instead of standard phrase structure trees.⁸ If the syntactic theory you are working with is of that kind, you may tacitly ignore the syntactic category labeling and linearization information encoded by the tree diagrams which we will be drawing in this book. It won’t make any difference.

Even though our semantics does not rely on syntactic category labels, we will normally employ at least some commonly used labels when we present phrase structure trees. Using syntactic category labels makes it easier for us to talk about particular nodes or types of nodes in a tree. We want to emphasize, however, that our choice of labels does not have any theoretical significance; the semantic component doesn’t have to see them. Here is an overview of the lexical category labels that we will be using most often:

syntactic category	label
verb	V
noun	N
adjective	A
preposition	P
determiner	D
inflectional elements (“do”, tense, modal auxiliaries, etc.)	I
complementizer	C

Labels for phrasal categories are coined in the usual way. A maximal verbal projection is a VP (verb phrase), for example. An intermediate verbal projection is a $\bar{V}$ (V-bar). We will be using “S” or “IP” (inflection phrase) as labels for sentences. Other labels may be introduced as we go along.

The kind of semantic theory we are developing here is compatible with a wide range of approaches to syntax. It may be worth pointing out, however, that it is nevertheless fundamentally incompatible not only with many conceivable proposals regarding the structure of the input to semantic interpretation, but even with some that have actually been made. For instance, Jackendoff⁹ and other representatives of the Extended Standard Theory argued that meaning depended on both Deep Structure and Surface Structure. Roughly, Deep Structure was to

determine predicate–argument relations, and Surface Structure scope and binding relations. In our terms, this would mean that the domain of the interpretation function $\llbracket \cdot \rrbracket$ should consist of (or include) something like *pairs* of phrase structure trees. What this would amount to in concrete detail is far from clear, and our current conception of semantic composition would have to be substantially altered and/or enriched to make it work.¹⁰

3.3 Well-formedness and interpretability

Montague's view of the role of syntax was that the syntactic component served to generate *exactly* the trees to which the semantic component assigned denotations. In such a framework, there is no such thing as a tree generated by the syntax but not in the domain of the interpretation function $\llbracket \cdot \rrbracket$, or a tree in the domain of $\llbracket \cdot \rrbracket$ but not generated by the syntax. Syntactic well-formedness and semantic interpretability coincide completely. This was just the situation that philosophical logicians were used to from their work on formal languages.

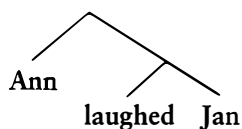
But linguists have questioned this set-up for a long time. Even before the word “modularity” became common, it was widely agreed that the grammars acquired by human beings do not just divide the set of all possible expressions into the well-formed, meaningful ones versus all the rest. Sentences or near-sentences may be judged deviant for a variety of independent reasons. It is easy to find examples that speakers reject as ungrammatical despite perfectly clear and definite intuitions about what they mean. Considering how general and universal our principles of interpretation are, of course, it is only to be expected that they will apply not only to those structures that happen to represent grammatical English sentences, but to many others besides.

The existence of the reverse case – that is, of uninterpretable but otherwise completely well-formed examples – is harder to establish without already presupposing a certain amount of theory, but there is certainly no reason to rule it out a priori either. The following examples might be cases in point.

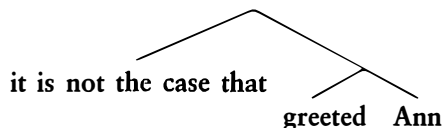
- (1) *Ann laughed Jan.
- (2) *It is not the case that greeted Ann.

Suppose the syntax of English provides derivations for these which are not ruled out by any syntactic (or phonological) principle and which assign them the following phrase structures.

(1')



(2')



What happens if we try to calculate denotations for these trees by means of our current composition rules and lexicon?

In (1') we obtain a truth-value for the constituent “laughed Jan” (by applying the function $\llbracket \text{laughed} \rrbracket$, of type $\langle e, t \rangle$, to Jan, of type e). But at the next higher node, we can't apply any of our rules. FA demands that either $\llbracket \text{Ann} \rrbracket$ or $\llbracket \text{laughed Jan} \rrbracket$ be a function. Since one is a person and the other a truth-value, it cannot apply. In (2'), we obtain a function of type $\langle e, t \rangle$ for the node dominating *greeted Ann* (by applying $\llbracket \text{greeted} \rrbracket$, of type $\langle e, \langle e, t \rangle \rangle$, to Ann, of type e). There again we are stuck at the next node up: $\llbracket \text{it is not the case that} \rrbracket$ is a function of type $\langle t, t \rangle$ and $\llbracket \text{greeted Ann} \rrbracket$ a function of type $\langle e, t \rangle$. Neither has the other in its domain, so FA cannot apply.

Thus, in both cases we are looking at trees that don't receive any denotation by our semantic component. That is, they are not in the domain of the $\llbracket \]$ function as defined by our current lexicon and composition rules. It is reasonable to hypothesize that this – and this alone – is what accounts for the deviance judgments that we represented by the asterisks in (1) and (2). We have not shown, of course, that there isn't something else wrong with these structures in addition to their failure to receive denotations. But let's suppose for the sake of the argument here that there isn't.

Structures which, like (1') and (2'), fail to receive denotations will be called *uninterpretable*. We take it that uninterpretability is one among other sources of ungrammaticality. Uninterpretable structures are those filtered out by the *semantic* component of the grammar. Here is a more precise formulation of our set of semantic rules, which explicitly takes into account the possible existence of uninterpretable (sub)trees. We define the interpretation function $\llbracket \]$ as the smallest function that fulfills the following conditions:

(3) *Terminal Nodes* (TN)

If α is a terminal node, then α is in the domain of $\llbracket \]$ if $\llbracket \alpha \rrbracket$ is specified in the lexicon.

(4) *Non-Branching Nodes* (NN)

If α is a non-branching node, and β is its daughter node, then α is in the domain of $\llbracket \cdot \rrbracket$ if β is. In this case, $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket$.

(5) *Functional Application* (FA)

If α is a branching node and $\{\beta, \gamma\}$ is the set of α 's daughters, then α is in the domain of $\llbracket \cdot \rrbracket$ if both β and γ are and $\llbracket \beta \rrbracket$ is a function whose domain contains $\llbracket \gamma \rrbracket$. In this case, $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket(\llbracket \gamma \rrbracket)$.

Notice that each of (3)–(5) gives sufficient (but not necessary) conditions for α being in the domain of the interpretation function $\llbracket \cdot \rrbracket$. But by defining $\llbracket \cdot \rrbracket$ as the *smallest* function that meets all these conditions, we say, in effect, that a tree is not in the domain of $\llbracket \cdot \rrbracket$ unless one of (3)–(5) implies that it is. We could have achieved the same result, of course, by using “iff” rather than “if” in (3)–(5).

Finally, let us make explicit the filtering function of the semantic component:

(6) *Principle of Interpretability*

All nodes in a phrase structure tree must be in the domain of the interpretation function $\llbracket \cdot \rrbracket$.

In sum, we are adopting a view of the grammar as a whole on which syntax and semantics are independent modules. Each imposes its own constraints on the grammatical structures of the language, and we expect there to be structures that are interpretable though syntactically illegitimate, as well as structures that are syntactically correct but uninterpretable.

3.4 The Θ -Criterion

In the syntactic literature, the ungrammaticality of examples like (1) and (2) has sometimes been attributed to the so-called Θ -Criterion (Theta-Criterion).

(1) *Ann laughed Jan.

(2) *It is not the case that greeted Ann.

Chomsky gives the following formulation.¹¹

(3) *Θ -Criterion*

Each argument bears one and only one Θ -role, and each Θ -role is assigned to one and only one argument.

What are θ -roles? The term " θ -role" (" θ -role") is an abbreviation for the more commonly used term "thematic role". If an event of greeting takes place somewhere, then there is a greeter and someone who is greeted. Greeter and greatee are two specific θ -roles. If a kicking takes place, then there is a kicker and someone who is kicked. Kicker and kickee are again two specific θ -roles. A verb like "greet" requires one argument that is interpreted as the greeter, and another argument that is interpreted as the one that is greeted. We may say, then, that it assigns these two specific θ -roles to its arguments. A verb like "kick" requires one argument that is interpreted as the kicker, and another argument that is interpreted as the one that is kicked. Again, we may say that it assigns those two specific θ -roles. Kicking and greeting are both actions, kickers and greeters are both agents, and whatever is greeted or kicked is a patient or theme according to a common terminology. Usually, when you hear about θ -roles, it's general θ -roles like "agent", "theme", "patient", or "experiencer" that are being talked about.¹¹ General θ -roles are potentially interesting, since there are generalizations linking them to syntactic positions in a systematic way. As far as the θ -criterion is concerned, however, only specific θ -roles are relevant. Now, what does the θ -criterion do?

One part of the θ -Criterion says that whenever a lexical element requires an argument with a certain θ -role, then there must be such an argument somewhere in the syntactic representation. And there can be only one. There couldn't be two NPs, for example, that are both interpreted as the greeter argument for the verb "greet". The other part of the θ -Criterion says that whenever there is a "candidate" for an argument, like an NP or what have you, then this element must be in fact an argument of a lexical element that assigns it a θ -role. That it can't be assigned more than one θ -role means that it can't fill more than one argument position at once.

The θ -Criterion is meant to exclude the following kinds of cases:

(4) *Ann laughed Jan. (= (1) above)

(5) *Greeted Ann.

(4) is ruled out by it since one of the NPs cannot be assigned a θ -role. "Laugh" has only one θ -role to assign, and since this one θ -role cannot be assigned to two NPs, (4) cannot possibly mean that Ann and Jan both laughed. (5) is ruled out by the θ -Criterion since "greeted" assigns two θ -roles, but there is only one NP present. Since no NP can receive two θ -roles, (5) cannot mean that Ann greeted herself.

We saw above how similar examples are treated by our current semantic theory: (4) came out as simply uninterpretable; that is, it receives no denotation.

In other words, it is predicted to have no truth-conditions at all, neither those of "Ann and Jan both laughed" nor any others. As far as (4) is concerned, then, the predictions of our semantics coincide fully with those of the Θ -Criterion.

May we conclude that the Θ -Criterion is – apart from differences in terminology – just a corollary of our current theory? In other words, does every Θ -Criterion violation reduce to a case of uninterpretability in the sense we specified above? Or is there merely a certain overlap in predictions? Let's take a closer look at this question.

What about (5)? The Principle of Interpretability succeeds (just like the Θ -Criterion) in ruling out examples like (2) above, in which (5) appears as an embedded constituent. But (5) all by itself isn't actually uninterpretable according to our semantics. It gets, in effect, the denotation of the VP "greeted Ann" (which is the function $\lambda x \lambda y \langle \lambda z, z \text{ greeted Ann} \rangle$). We therefore predict correctly that "greeted Ann" cannot be used to make a statement – that is, is not capable of truth or falsity.

Summarizing the discussion so far, we still haven't seen any predictions of the Θ -Criterion that were not covered by the Principle of Interpretability in our current theory. But it would be premature to stop looking. In fact, the following abstract consideration shows that the Θ -Criterion is a genuinely stronger constraint than interpretability.

Suppose we have a predicate α with one Θ -role to assign. In our terms, suppose that $[\alpha]$ is of type $\langle \alpha, \tau \rangle$. According to the Θ -Criterion, α must appear in the vicinity of something which receives its Θ -role. That means α has to have a sister node with a meaning of type τ . According to our Interpretability principle, on the other hand, a sister node of type τ is not strictly required. It would provide one suitable environment for α , but not the only kind. Imagine instead that α has a sister node whose meaning is a function with domain $D_{\langle \alpha, \tau \rangle}$. (For instance, it might be of type $\langle \alpha, \tau, \tau \rangle$.) In that case, the next higher node could be interpreted by applying this sister's meaning to $[\alpha]$. So we could have an interpretable structure which does not contain an argument for α if α would not be assigning its Θ -role to any phrase, in violation of the Θ -Criterion. Yet Interpretability would be fulfilled, in virtue of α being a suitable argument for its sister node.

The question now is: Can we find concrete instances of this sort of situation? And if yes, are they in fact grammatical (as predicted by Interpretability) or ungrammatical (as predicted by the stronger Θ -Criterion)?

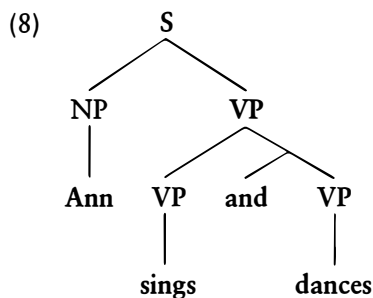
In the following chapter, we will propose that common nouns like "barn" are 1-place predicates (type $\langle \alpha, \tau \rangle$). In other words, they have a Θ -role to assign, and thus the Θ -Criterion requires the presence of an argument. In certain examples (predicative uses), this is unproblematic:

The required argument here is the subject NP “this”. (6) is true if and only if the object referred to by “this” has the property of being a barn. But consider the following sentence:

(7) The barn burned down.

(7) contains no phrase that receives the Θ -role of “barn”. It thus seems to violate the Θ -Criterion. Yet it is perfectly fine, and we will see below how it can be interpreted by assigning “the” a meaning of type $\langle\langle e, t \rangle, e \rangle$, suitable to take $\llbracket \text{barn} \rrbracket$ as an argument. So this is the sort of case we have been looking for. The Interpretability Principle and the Θ -Criterion make different predictions here, and if the analysis we will give for (7) is on the right track, then the empirical facts favor the former.¹³

Another case which *prima facie* points in the same direction arises with coordinated predicates. So far, all our examples involving “and” and “or” had these connectives combining with sentences, as in propositional logic. But English apparently also allows these connectives to conjoin subsentential constituents, such as two VPs, as in (8).¹⁴



To interpret this, we need a new lexical entry for the homonym of “and” that we encounter here. (The familiar meaning of type $\langle t, \langle t, t \rangle \rangle$ evidently doesn’t work here.)

(9) $\llbracket \text{and} \rrbracket = \lambda f \in D_{\langle e, t \rangle} \cdot [\lambda g \in D_{\langle e, t \rangle} \cdot [\lambda x \in D_e \cdot f(x) = g(x) = 1]]$

You can verify that, given (9), the tree in (8) is interpretable by our general principles and receives intuitively correct truth-conditions. What interests us here is that this is another interpretable structure which seems to violate the Θ -Criterion, in that there are not enough arguments to go around for all the Θ -roles that need to be assigned in (8). “Sing” and “dance” each have a Θ -role to assign, but only one potential argument (the NP “Ann”) is present. Once more, we tentatively conclude that the weaker requirements imposed by our Interpretability Principle make the better empirical predictions.

These two arguments against the Θ -Criterion are not beyond question, of course. They are only as good as the syntactic and semantic analyses we have sketched. It might very well turn out upon closer inspection that there is more than meets the eye to the structures of examples (7) and (8). Specifically, we might find evidence for non-overt constituents which provide just the superficially missing arguments that are demanded by the Θ -Criterion. In that event, these examples might not only cease to be counterexamples, but might ultimately turn out even to support the Θ -Criterion. We will set this possibility aside for now, but in principle it remains open.

3.5 Argument structure and linking

Some syntactic theories posit a syntactic representation of a verb's argument structure that is distinct from the representation of the verb's denotation. *Argument structure* representations are meant to encode "the syntactically relevant argument-taking properties of a verb"¹⁵ (and any lexical item that takes arguments). Argument structure representations play a role in theories of *linking* – that is, theories about how a verb's arguments are linked to syntactic positions in a tree. In this section, we will look at some proposals that have actually been made for argument structure representations and linking. We will see that some of the information that has been attributed to argument structure representations and linking principles turns out to be redundant, given the semantic theory we have been developing.

Minimally, the argument structure representations found in the syntactic literature list the arguments a verb takes. This information has been thought to be relevant to the syntax, because of the deviance of sentences like (1) or (2):

- (1) *Ann laughed Jan.
- (2) *Greeted Ann.

In the previous section, we discussed the possibility that the deviance of (1) and (2) might actually be accounted for in the semantics. (1) fails Interpretability, and (2) receives a VP-denotation, hence cannot be used to make a statement capable of truth or falsity. If this view of the syntax–semantics interface should turn out to be correct, there is no need for separate syntactic representations of argument structure, unless they provide more information than a mere list of the verb's arguments.

In her book *Argument Structure*,¹⁶ Jane Grimshaw explores the hypothesis that argument structure representations also reflect prominence relations among arguments. The verb “introduce” would be given the following argument structure representation, for example:¹⁷

(3) introduce (agent (goal (theme)))

(3) says that “introduce” has three arguments that are hierarchically ordered: the agent argument is the highest, the goal argument comes next, and the theme argument is at the bottom. Grimshaw emphasizes that the general thematic role labels “agent”, “goal”, or “theme” have no theoretical status; they merely serve to identify the verb’s arguments. She could have used labels for specific thematic roles instead. A prominence relation among arguments is part and parcel of our Fregean verb denotations. The Fregean denotation of “introduce” is

(4) $\lambda x \in D_e . [\lambda y \in D_e . [\lambda z \in D_e . z \text{ introduces } x \text{ to } y]]$

Like (3), (4) encodes the information that the agent argument of “introduce” is most prominent, the goal argument is next, and the theme argument fills the lowest position. If we apply the function in (4) to an individual, Sue, for example, we get (5).

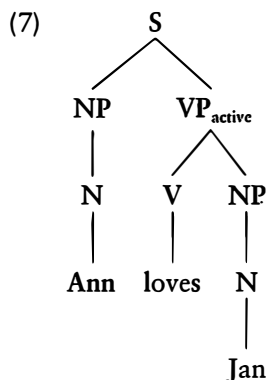
(5) $\lambda y \in D_e . [\lambda z \in D_e . z \text{ introduces Sue to } y]$

The first argument to be processed, then, is given the role of the person who is introduced (the theme). Next, if we apply the function in (5) to Ann, the result is (6): Ann is assigned the role of the person who Sue is introduced to (the goal):

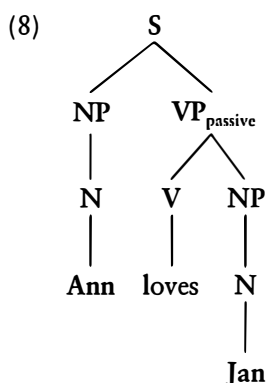
(6) $\lambda z \in D_e . z \text{ introduces Sue to Ann}$

If we finally apply the function in (6) to, say, Pat, we end up with a truth-value: “True” if Pat introduces Sue to Ann, and “False” otherwise. That is, the last argument to be processed is understood as the agent of the introduction event. We can conclude, then, that we do not need separate argument structure representations to display prominence relations among arguments. This information is already provided by the representation of the verb’s denotation.

Our system of type-driven interpretation principles implies a rather strong claim about the linking of a verb’s arguments to syntactic positions. The lexically determined prominence relations must be preserved in the syntax.¹⁸ This means that there couldn’t be a natural language that has structures like (7) *as well as* structures like (8), with the truth-conditions (7’) and (8’) respectively.



- (7') *Truth-conditions*
 $\llbracket 7 \rrbracket = 1$ iff Ann loves Jan.



- (8') *Truth-conditions*
 $\llbracket 8 \rrbracket = 1$ iff Jan loves Ann.

Since the V-denotation must combine with the denotation of its direct object by Functional Application, the lexical meaning of “loves” determines that “Jan” is interpreted as the one who is loved in both (7) and (8). Consequently, (7) and (8) cannot have different truth-conditions. If we allowed construction-specific interpretation rules, we could in principle have rules like (9) and (10):

- (9) If α has the form $\begin{array}{c} \text{VP}_{\text{active}} \\ \swarrow \quad \searrow \\ \beta \quad \gamma \end{array}$, then $\llbracket \alpha \rrbracket = \llbracket \beta \rrbracket (\llbracket \gamma \rrbracket)$.

- (10) If α has the form $\begin{array}{c} \text{VP}_{\text{passive}} \\ \swarrow \quad \searrow \\ \beta \quad \gamma \end{array}$, then $\llbracket \alpha \rrbracket = [\lambda x \in D_e \quad \llbracket \beta \rrbracket (x) (\llbracket \gamma \rrbracket)]$.

Keeping our earlier rules for the interpretation of lexical, non-branching, and S-nodes, both (7) and (8) would now be assigned the intended truth-conditions.

Active structure (7)

$\llbracket \text{VP}_{\text{active}} \rrbracket = [\lambda y \in D_e . [\lambda z \in D_e . z \text{ loves } y]](\text{Jan}) = [\lambda z \in D_e . z \text{ loves Jan}].$
 $\llbracket 7 \rrbracket = [\lambda z \in D_e . z \text{ loves Jan}](\text{Ann}) = 1 \text{ iff Ann loves Jan}.$

Passive structure (8)

$\llbracket \text{VP}_{\text{passive}} \rrbracket = [\lambda x \in D_e . [\lambda y \in D_e . [\lambda z \in D_e . z \text{ loves } y]](x)(\text{Jan})]$
 $= [\lambda x \in D_e . \text{Jan loves } x].$
 $\llbracket 8 \rrbracket = [\lambda x \in D_e . \text{Jan loves } x](\text{Ann}) = 1 \text{ iff Jan loves Ann}.$

Many syntactic theories consider it necessary to stipulate principles that prevent a language from having *both* structures like (7) *and* structures like (8). Take Mark Baker's Uniformity of Theta Assignment Hypothesis (UTAH):¹⁹

(11) *The Uniformity of Theta Assignment Hypothesis (UTAH)*

Identical thematic relationships between items are represented by identical structural relationships.

The UTAH can be given a weaker or stronger interpretation. On the weaker interpretation, the UTAH says that NPs that bear the same *specific* thematic role bear the same syntactic relationship to their verb. The stronger interpretation requires that all NPs that bear the same *general* thematic role (given some inventory of general thematic roles) bear the same syntactic relationship to their verb. The weaker version of the UTAH requires that all arguments that bear the lover role must be syntactically realized in a uniform way, and the same is true of all arguments that bear the role of the one who is loved. The stronger version of the UTAH might require, for example, that all arguments that bear the experiencer (of an emotion) role appear in the same syntactic position, and likewise all arguments bearing the theme, patient, or agent role. Both versions exclude the hypothetical situation described above. If (7') states the correct truth-conditions for (7), then the truth-conditions (8') are ruled out for (8), and if (8') states the correct truth-conditions for (8), then the truth-conditions (7') are ruled out for (7).

We have just seen that some of the work done by principles like the UTAH is automatically taken care of by our semantic component. In fact, the weak version of the UTAH is at least close to becoming superfluous. What about the more interesting strong version? What about *general* thematic roles in our framework? It is fairly obvious that the prominence relations among a verb's arguments do not have to be learned separately for each verb. There are generalizations involving general thematic roles. Agent arguments are generally higher than theme or patient arguments, for example. The exact nature of such generalizations is still a matter of debate. The most worked-out proposal is that of Dowty,²⁰

whose argument selection principles are stated on the basis of thematic proto-roles. Dowty assumes that thematic roles like agent or patient are cluster concepts like the prototypes of Rosch and Mervis.²¹ According to him, the argument with the greatest number of proto-agent properties is selected as the lexically most prominent argument, for example. With ditransitive verbs, the argument with the greatest number of proto-patient properties would be the lowest argument. The middle argument would have fewer proto-patient properties than the lowest argument and fewer proto-agent properties than the highest argument. Be this as it may, whatever the correct generalizations about lexical prominence relations are, our semantic interpretation system automatically imposes them on the hierarchical line-up of arguments in the syntax. The syntactic component, then, does not have to worry about thematic roles, be they specific or general. What appear to be generalizations about the syntactic realization of arguments might in fact be rooted in uniformities of prominence relations across lexical items.

Does this mean that we can dispense with syntactic linking principles altogether? Not quite yet. There is an important syntactically relevant distinction between a verb's arguments that does not yet fall out from our semantics. Syntactic work that was most vigorously pursued within Relational Grammar has established that not all arguments that are lexically most prominent show the same syntactic behavior.²²

- (12) Unaccusative verb: $\llbracket \text{die} \rrbracket = \lambda x \in D_e . x \text{ dies}$
 Unergative verb: $\llbracket \text{work} \rrbracket = \lambda x \in D_e . x \text{ works}$
- (13) Transitive agentive verb: $\llbracket \text{greet} \rrbracket = \lambda x \in D_e . [\lambda y \in D_e . y \text{ greets } x]$
 Transitive experiencer verb: $\llbracket \text{worry} \rrbracket = \lambda x \in D_e . [\lambda y \in D_e . y \text{ worries } x]$

As far as lexical prominence relations are concerned, there is no difference between "die" and "work", and the same is true for "greet" and "worry". Yet syntactically, unaccusative verbs behave differently from unergative verbs, and agentive verbs do not pattern with (object) experiencer verbs in important respects. Using the terminology of Williams,²³ the most prominent argument of unergative and agentive transitive verbs is an *external* argument. The most prominent argument of unaccusative and (object) experiencer verbs is an *internal* argument. According to Williams, the external argument is located external to the maximal projection of the verb, whereas internal arguments appear within the maximal projection of the verb (at some level of representation). In one way or other, the difference between external and internal arguments has been held responsible for the fact that subjects of unaccusative and (object) experiencer verbs show certain properties of objects, unlike the subjects of unergative and agentive transitive verbs.

The syntactic impact of the distinction between external and internal arguments cannot be directly derived from our semantics as is. To be sure, there are plenty of proposals which try to correlate the distinction with some semantic property. But that's not enough. This all by itself will not spare us the stipulation of a special linking principle. Our semantic composition rules only impose hierarchical relationships on the syntactic representation of arguments. There is no way for them to influence whether a most prominent argument is realized within or outside a VP, for example. One possible conclusion, then, is that the distinction between external and internal arguments might be the only piece of information about a verb's argument structure that has to be taken care of in the syntax. This is not a necessary conclusion, however. Marantz²⁴ has argued that external arguments are not arguments of their verbs at all. Within our current semantic framework, it is hard to see how to even make sense of such an idea. It is possible, however, to implement it within a theory that construes verb denotations along the lines of Donald Davidson.²⁵ Kratzer shows how this move eliminates the need for a syntactic representation of the distinction between external and internal arguments.²⁶ All information about a verb's argument structure, then, would be directly derivable from its denotation. There would be no syntactic theory of argument structure or linking. While this line of research might have promise, it goes way beyond what we can and should pursue in an introductory text.

Notes

- 1 This is similar to the method of "rule-by-rule interpretation" (so dubbed by Emmon Bach) which you find in Montague's papers, for example. Suppose we described the set of phrase structures that are interpretable by our current semantics by means of a context-free grammar. Then each of our semantic rules would correspond to exactly one of the phrase structure rules – hence the name "rule-by-rule". See E. Bach, "An Extension of Classical Transformational Grammar," in *Problems of Linguistic Metatheory*, Proceedings of the 1976 Conference at Michigan State University, 1976, pp. 183–224.
- 2 E. Klein and I. Sag, "Type-Driven Translation," *Linguistics and Philosophy*, 8 (1985), pp. 163–201. For a recent approach along similar lines, see M. Bittner, "Cross-Linguistic Semantics," *Linguistics and Philosophy*, 17 (1994), pp. 53–108. Bittner uses an intensional semantics and covers a much greater range of constructions than we do here.
- 3 Note that we can't write " $\lambda p \in D_t . [\lambda q \in D_t . p \text{ and } q]$ ". "p" and "q" are variables for truth-values, so the "and" in "p and q" would have the same status as the "and" in "1 and 0", for example. Without further conventions, it is not clear what this would mean.
- 4 This has been argued for, e.g., by R. Kayne, *Connectedness and Binary Branching* (Dordrecht, Foris, 1984).

- 5 See B. H. Partee, A. ter Meulen, and R. E. Wall, *Mathematical Methods in Linguistics* (Dordrecht, Kluwer, 1990), chs 16–22, for definitions of this and other types of formal grammars.
- 6 The question whether and, if yes, what kind of phrase structure trees are actually constructed during sentence processing has been at the center of debates in psycholinguistics. See L. Frazier, "Sentence Processing: A Tutorial Review," in M. Coltheart (ed.), *Attention and Performance*, vol. 12 (Hillsdale, NJ, Lawrence Erlbaum Associates, 1987), pp. 559–86. Also J. D. Fodor, "Comprehending Sentence Structure," and M. Steedman, "Computational Aspects of the Theory of Grammar," in L. R. Gleitman and M. Liberman (eds), *Language*, vol. 1 of D. N. Osherson (ed.), *An Invitation to Cognitive Science*, 2nd edn (Cambridge, Mass., MIT Press, 1995), pp. 209–46 and 247–81 respectively.
- 7 For a precise definition, see e.g., Partee et al., *Mathematical Methods*, pp. 443–4.
- 8 The phrase markers contemplated by Chomsky in recent work, for instance, don't have linear order. They do have syntactic category labels, though. See N. Chomsky, "A Minimalist Program for Linguistic Theory," in K. Hale and S. J. Keyser (eds), *The View from Building 20* (Cambridge, Mass., MIT Press, 1993), pp. 1–52.
- 9 R. Jackendoff, *Semantic Interpretation in Generative Grammar* (Cambridge, Mass., MIT Press, 1972).
- 10 Similar comments apply to N. Hornstein, *Logic as Grammar* (Cambridge, Mass., MIT Press, 1984).
- 11 N. Chomsky, *Lectures on Government and Binding* (Dordrecht, Foris, 1981), p. 36.
- 12 An important modern reference is J. S. Gruber, "Studies in Lexical Relations" (Ph.D. dissertation, MIT, 1965). See also J. S. Gruber, *Lexical Structures in Syntax and Semantics* (Amsterdam, North-Holland, 1976).
- 13 We might stress that this objection is directed at the specific formulation of the Θ -Criterion that we have quoted. It does not apply to all versions of the Θ -Criterion that you find in the literature. James Higginbotham, in particular, has proposed to replace the second part of the Θ -Criterion by the more flexible requirement that every thematic role must somehow be *discharged*. See J. Higginbotham, "On Semantics," *Linguistic Inquiry*, 16 (1985), pp. 547–93; *idem*, "Elucidations of Meaning," *Linguistics and Philosophy*, 12 (1989), pp. 465–517. In the latter he spells out a list of different discharge mechanisms (pp. 475f.).
- 14 Recall our pedagogical decision to avoid ternary branching – hence the somewhat arbitrary right-branching structure.
- 15 B. Levin and M. Rappaport Hovav, *Unaccusativity. At the Syntax–Lexical Semantics Interface* (Cambridge, Mass., MIT Press, 1995), p. 21.
- 16 J. Grimshaw, *Argument Structure* (Cambridge, Mass., MIT Press, 1990).
- 17 *Ibid.*, p. 4, with "announce" instead of "introduce".
- 18 This is actually not *quite* true. Our semantics doesn't force us to assume that there is *no* way to change the argument structure of a verb. It does imply, however, that if there are such changes at all, they have to be brought about by specific morphemes. We *could* have a morpheme of the following kind, for example:

$[\text{passive}] = \lambda f \in D_{\langle e, \langle e, t \rangle \rangle} . [\lambda x \in D_e . \text{there is some } a \in D_e \text{ such that } f(x)(a) = 1]$

$[\text{passive}]$ operates on transitive V- or $\bar{V}$ -denotations, and eliminates the highest argument. An analysis of this kind for the participle affix "–en" was proposed by D. Dowty, "Governed Transformations as Lexical Rules in a Montague Grammar," *Linguistic Inquiry*, 9 (1978), pp. 393–426. We will not take a stand here as to

whether natural languages have items like passive. All we are interested in right now is to point out that our framework requires any change in argument structure to be brought about by some morpheme.

- 19 M. C. Baker, *Incorporation. A Theory of Grammatical Function Changing* (Chicago, University of Chicago Press, 1988), p. 46. Baker's condition includes the requirement that UTAH is a principle for Deep Structure representations. At the present stage of our investigation, we are not presupposing a multistratal syntax, so we left out the level-specification in Baker's original formulation of the UTAH. Baker's UTAH is a stronger version of Perlmutter and Postal's Universal Alignment Hypothesis (UAH), which states that "there exist principles of universal grammar which predict the initial relation borne by each nominal in a given clause from the meaning of the clause" (D. M. Perlmutter and P. M. Postal, "The 1-Advancement Exclusiveness Law," in D. M. Perlmutter and C. G. Rosen (eds), *Studies in Relational Grammar*, vol. 2 (Chicago, University of Chicago Press, 1984), pp. 81–125). Principles like UAH or UTAH are defended in a very sophisticated way in D. Pesetsky, *Zero Syntax, Experiencers and Cascades* (Cambridge, Mass., MIT Press, 1995), but have been argued to be untenable by Carol Rosen, "The Interface between Semantic Roles and Initial Grammatical Relations," in Perlmutter and Rosen (eds), *Relational Grammar*, pp. 38–77.
- 20 D. Dowty, "Thematic Proto-Roles and Argument Selection," *Language*, 67/3 (1991), pp. 547–619.
- 21 E. Rosch and C. B. Mervis, "Family Resemblances: Studies in the Internal Structure of Categories," *Cognitive Psychology*, 8 (1975), pp. 382–439.
- 22 A good collection of relevant work is found in the three volumes of *Studies in Relational Grammar* published by the University of Chicago Press: vol. 1, ed. D. M. Perlmutter (1983); vol. 2, ed. D. M. Perlmutter and C. G. Rosen (1984); vol. 3, ed. P. M. Postal and B. D. Joseph (1990). For unaccusativity, consult also Levin and Rappaport Hovav, *Unaccusativity*. For experiencer verbs, an important recent reference is A. Belletti and L. Rizzi, "Psych-verbs and Θ -theory," *Natural Language and Linguistic Theory*, 6 (1988), pp. 291–352. See also the discussions in Grimshaw, *Argument Structure*, and Pesetsky, *Zero Syntax*.
- 23 E. Williams, "Argument Structure and Morphology," *Linguistic Review*, 1 (1981), pp. 81–114.
- 24 A. Marantz, *On the Nature of Grammatical Relations* (Cambridge, Mass., MIT Press, 1984).
- 25 D. Davidson, "The Logical Form of Action Sentences," in N. Rescher (ed.), *The Logic of Decision and Action* (Pittsburgh, University of Pittsburgh Press, 1967), pp. 81–95. See also the neo-Davidsonian verb meanings of T. Parsons, *Events in the Semantics of English. A Study in Subatomic Semantics* (Cambridge, Mass., MIT Press, 1990).
- 26 A. Kratzer, "Severing the External Argument from its Verb," in J. Rooryck and L. Zaring (eds), *Phrase Structure and the Lexicon* (Dordrecht, Kluwer Academic Publishers, 1996), pp. 109–37.